



Tooling

Grundlagen Software Entwicklung

Autor	Jens Reiner
Institution	DHBW Heidenheim - Wirtschaftsinformatik
Jahr	2026
Version	1.0.0

Tooling

Version 1.0.0 · Stand 2026-10-04 · Commit 82c4d7a

© 2026 Jens Reiner · Lizenz: CC BY-SA 4.0



Inhalt

Teil 1 • Tooling	1
1.1 Dateisystem & Pfade	2
1.2 VS Code	11
1.3 Browser & DevTools	16
1.4 Git & GitHub	23
1.5 Markdown & README	32
1.6 Git in der Gruppe	38
Anhang	48
A Glossar	49



Teil 1 • Tooling



1.1 Dateisystem & Pfade

Voraussetzungen: keine

Lernziele

- Dateiendungen und versteckte Dateien einblenden und den Ordner projekte außerhalb der Cloud-Synchronisation anlegen.
- Die Konventionen für Datei- und Ordernamen in Webprojekten anwenden und die Folge eines Verstoßes in der Adresse nachweisen.
- Relative Pfade lesen und schreiben und von absoluten Pfaden unterscheiden.

1. Vorschau

Zielstand nach diesem Kapitel:

```
projekte/
├── pfad-training/           ← Übungsmaterial für die Tooling-Kapitel
│   ├── index.html
│   ├── pages/ ...
│   └── assets/ ...
└── toolbox/                ← Lernprojekt dieses Skripts
    └── learning-notizen.md ← Lernprotokoll
```

2. Warum

Eine Website besteht aus Dateien, die einander über Pfade referenzieren: HTML-Dateien mit dem Inhalt der Seiten, Stylesheets mit der Gestaltung und Bilder. Viele Fehler in den ersten Projekten entstehen im Dateisystem und nicht im HTML. Beispiele sind eine falsche Dateiendung, ein Pfad mit einer Ebene zu viel und eine Datei im falschen Ordner.

Weiterführend: [MDN – Dealing with files](#) · [MDN – What is a URL?](#)

3. Einrichten

Dieses Kapitel verwendet den Dateimanager des Betriebssystems (Windows: **Explorer**, macOS: **Finder**) und den Browser Chrome.

3.1 Dateiendungen und versteckte Dateien einblenden

Windows und macOS blenden Dateiendungen standardmäßig aus. Eine Datei mit dem angezeigten Namen `index.html` kann deshalb `index.html.txt` heißen. Der Browser stellt sie dann als Text dar.

Dateien und Ordner, deren Name mit einem Punkt beginnt, sind ebenfalls ausgeblendet. Beispiele sind `.gitignore` und der Ordner `.git`. Die folgenden Kapitel verwenden solche Dateien.

Einstellung	Windows 11	macOS
Dateiendungen	Explorer → Anzeigen → Einblenden → Dateinamenerweiterungen	Finder → Einstellungen (Cmd + ,) → Erweitert → Alle Dateinamensuffixe einblenden
Versteckte Dateien	Explorer → Anzeigen → Einblenden → Ausgeblendete Elemente	Im Finder Cmd + Shift + . (schaltet um)

3.2 Ordner projekte anlegen

Lege im **Benutzerordner** den Ordner `projekte` an. Er enthält alle Projekte dieses Skripts.

	Windows	macOS
Benutzerordner öffnen	%USERPROFILE% in die Adressleiste des Explorers eingeben	Finder → Gehe zu → Benutzerordner (Cmd + Shift + H)
Pfad anzeigen	Klick in die Adressleiste	Darstellung → Pfadleiste einblenden (Cmd + Option + P)
Beispielpfad	C:\Users\anna\projekte	/Users/anna/projekte

Mit eingblendeten versteckten Dateien zeigt der Benutzerordner den Ordner `AppData` (Windows) bzw. `Library` (macOS). Fehlt der Ordner, ändere die Einstellung aus [3.1](#); unter macOS drückst du Cmd + Shift + . noch einmal.

Die Ordner „Desktop“ und „Dokumente“ sind als Ablageort ungeeignet. Sie sind häufig mit OneDrive (Windows) oder iCloud (macOS) synchronisiert. Die Synchronisation sperrt Dateien während der Arbeit und erzeugt Konfliktkopien. Beides stört die Versionskontrolle mit Git. Zusätzlich enthalten diese Pfade oft Leerzeichen.

3.3 Namensregeln für Dateien und Ordner

Die folgenden Regeln gelten für alle Dateien und Ordner eines Webprojekts. Die Schreibweise heißt **kebab-case**.

Regel	Richtig	Falsch
nur Kleinbuchstaben	<code>logo.svg</code>	<code>Logo.SVG</code>
Bindestrich statt Leerzeichen	<code>ueber-uns.html</code>	<code>Über uns.html</code>
keine Umlaute, kein ß, keine Sonderzeichen	<code>groesse.css</code>	<code>größe(2).css</code>
Name beschreibt den Inhalt	<code>kontakt.html</code>	<code>seite3_final_NEU.html</code>

Begründung:

- Jeder Dateiname wird Teil einer **URL**, der Adresse im Browser. Eine URL stellt Leerzeichen und Umlaute nicht unverändert dar.

- Windows und macOS unterscheiden in Dateinamen nicht zwischen Groß- und Kleinschreibung. Webserver, also Rechner, die eine Website ausliefern, laufen überwiegend unter Linux und unterscheiden Groß- und Kleinschreibung. Ein Pfad auf Logo.svg findet die Datei Logo.svg lokal, auf dem Server aber nicht.
- Im Terminal müssen Namen mit Leerzeichen in Anführungszeichen stehen. Das Terminal führt das Kapitel VS Code (Kapitel 1.2) ein.

3.4 Toolbox und Lernprotokoll anlegen

Die Toolbox ist das Lernprojekt dieses Skripts: eine Website, die mit jedem Kapitel wächst. Das Lernprotokoll learning-notizen.md liegt im selben Ordner. Es enthält zu jedem Experiment die Vorhersage, das Ergebnis und die Erklärung. Ein **Predict** fragt, was passiert, bevor du es ausführst; ein **Break-it** macht etwas absichtlich kaputt.

1. Lege in projekte den Ordner toolbox an.
2. Lege im Ordner toolbox die Datei learning-notizen.md an.

Windows

1. Öffne den Ordner toolbox und wähle mit einem Rechtsklick auf eine freie Fläche **Neu** → **Textdokument**.
2. Benenne die Datei in learning-notizen.md um.
3. Erscheint eine Warnung zur Dateinamenerweiterung, bestätige sie mit **Ja**.
4. Öffne die Datei mit Rechtsklick → **Öffnen mit** → **Editor**.

macOS

1. Öffne TextEdit. Erscheint ein Fenster zum Öffnen, klicke auf **Neues Dokument**.
2. Wähle **Format** → **In reinen Text umwandeln** (Cmd + Shift + T).
3. Sichere das Dokument als learning-notizen.md. Wähle als Ort mit Cmd + Shift + H den Benutzerordner und darin projekte → toolbox.
4. Wähle im Dialog „.md“ verwenden.

3. Übernimm diesen Inhalt, ersetze (Datum) durch das aktuelle Datum und speichere die Datei.

```
# Learning-Notizen

Lernprotokoll: je Experiment Vorhersage, Ergebnis und Erklärung.

## Dateisystem & Pfade (Datum)

### Beobachtung: Die drei Seiten
- **Ergebnis:**
- **Erklärung:**

### Predict 1: Die Adresszeile
- **Vorhersage:**
- **Ergebnis:**
- **Erklärung:**
```

4. Prüfe den Dateinamen im Dateimanager. Er lautet learning-notizen.md, ohne angehängtes .txt oder .rtf. Steht dort learning-notizen.md.txt, benenne die Datei um und lösche .txt. Steht dort .rtf, lösche die Datei und lege sie wie in Schritt 2 neu an.

Die Zeichen # und - gehören zu Markdown.

3.5 Material entpacken

Lade [pfad-training.zip](#) herunter und entpacke die Datei in den Ordner **projekte**. Der Browser speichert die Datei im Ordner **Downloads**.

Achtung (Windows): Ein Doppelklick öffnet die ZIP-Datei, entpackt sie aber nicht. Seiten, die aus der geöffneten ZIP-Datei starten, finden ihre Stylesheets und Bilder nicht.

Windows	macOS
Rechtsklick auf die ZIP-Datei → Alle extrahieren... → als Ziel den Ordner projekte eintragen, zum Beispiel <code>C:\Users\anna\projekte</code> , ohne <code>pfad-training</code> am Ende	Safari entpackt die ZIP-Datei beim Herunterladen; in Downloads liegt dann der Ordner <code>pfad-training</code> . Liegt dort die ZIP-Datei, entpacke sie mit einem Doppelklick. Verschiebe den Ordner <code>pfad-training</code> nach projekte .

Prüfe das Ergebnis: Die Datei `index.html` liegt direkt in `projekte/pfad-training`, nicht in einem zweiten Unterordner `pfad-training`. Liegt sie in `pfad-training/pfad-training`, verschiebe den Inhalt des inneren Ordners eine Ebene nach oben und lösche den leeren inneren Ordner.

Spätere Kapitel verweisen auch auf einzelne Dateien aus dem Material. Der Browser speichert sie ebenfalls in **Downloads**. Verschiebe eine solche Datei an den genannten Ort. Liegt dort schon eine Datei mit demselben Namen, bestätige das Ersetzen.

3.6 Chrome installieren

Dieses Skript verwendet Chrome als Referenzbrowser. Lade die Installationsdatei von [google.com/chrome](https://www.google.com/chrome) herunter.

Windows	macOS
Starte die heruntergeladene Datei, bestätige die Abfrage der Berechtigung mit Ja und folge der Anleitung auf dem Bildschirm.	Öffne <code>googlechrome.dmg</code> und ziehe Chrome in den Ordner Programme . Starte Chrome aus Programme und bestätige die Rückfrage von macOS mit Öffnen .

3.7 Datei im Browser öffnen

1. Öffne Chrome.
2. Öffne `projekte/pfad-training/index.html` mit `Strg + O` (macOS: `Cmd + O`). Unter macOS wählst du im Dialog mit `Cmd + Shift + H` den Benutzerordner.

Chrome zeigt die Startseite: oben das Logo, daneben die Links **Start**, **Über** und **Kontakt** in einer Zeile, darunter die farbige Überschrift „Pfad-Training“.

3.8 Tastenkürzel

Aktion	Windows	macOS
Datei in Chrome öffnen	<code>Strg + O</code>	<code>Cmd + O</code>
Kopieren	<code>Strg + C</code>	<code>Cmd + C</code>
Einfügen	<code>Strg + V</code>	<code>Cmd + V</code>
Benutzerordner im Finder oder im Dialog zum Öffnen	–	<code>Cmd + Shift + H</code>
Versteckte Dateien im Finder ein- und ausblenden	–	<code>Cmd + Shift + .</code>

4. Verstehen

Lege im Lernprotokoll für jede weitere Aufgabe einen Abschnitt wie **### Predict 1: Die Adresszeile** an, zum Beispiel **### Predict 2: Relative Pfade**. Nimm jede Änderung nach einem Experiment zurück, bevor das nächste beginnt.

Der Ablauf der Beobachtung ist: Aufgabe ausführen, Ergebnis und Erklärung notieren.

Beobachtung: Die drei Seiten

- **Aufgabe:** Rufe von der Startseite aus die Seiten **Start**, **Über** und **Kontakt** auf. Zwei der drei Seiten enthalten einen Fehler.
- **Notieren:** die sichtbaren Abweichungen je Seite als Ergebnis. Trage die Erklärung für **Über** nach Predict 2 nach. Die Ursache auf **Kontakt** klärt das Kapitel Browser & DevTools (Kapitel 1.3).

→ Auflösung 1 am Ende des Kapitels

Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Predict 1: Die Adresszeile

Die Startseite `index.html` ist in Chrome geöffnet. Es gibt keinen Server und keine Domain.

- **Vorhersage:** Welche Adresse zeigt die Adresszeile des Browsers? Verwendet sie unter Windows \ oder /?
- **Prüfen:** Klicke in die Adresszeile und lies die vollständige Adresse.

→ Auflösung 2 am Ende des Kapitels

Predict 2: Relative Pfade

Aufbau des Pfad-Trainings:

```
pfad-training/  
├── index.html  
├── pages/  
│   ├── about.html  
│   └── team/  
│       └── kontakt.html  
└── assets/  
    ├── css/  
    │   └── style.css  
    └── img/  
        └── logo.svg
```

Ein **relativer Pfad** beschreibt den Weg von der Datei aus, in der er steht. Ein Pfad ohne / am Anfang beginnt im Ordner der aktuellen Datei. `ordner/datei` führt in einen Ordner hinein, `../` führt eine Ebene nach oben.

- **Vorhersage:** Die Spalten **Von** und **Nach** nennen die Dateien vom Ordner `pfad-training` aus. Notiere für jede Zeile den relativen Pfad von der Datei in **Von** zur Datei in **Nach**.

#	Von	Nach
1	index.html	assets/css/style.css
2	pages/about.html	assets/img/logo.svg
3	pages/team/kontakt.html	index.html
4	pages/team/kontakt.html	pages/about.html
5	assets/css/style.css	assets/img/logo.svg

- **Prüfen:** Vergleiche mit der Auflösung.

→ [Auflösung 3](#) am Ende des Kapitels

Break-it 1: Eine Datei umbenennen

Benenne pages/about.html in über uns.html um. Der Name endet auf genau ein .html.

- **Vorhersage:**
 1. Was zeigt der Browser nach einem Klick auf **Über** auf der Startseite?
 2. Welche Adresse zeigt der Browser, wenn du über uns.html in Chrome öffnest?
 3. Wie heißt die Datei Anfahrt Büro.HTML nach den [Namensregeln](#)?
- **Prüfen:** Führe 1 und 2 aus. Kopiere die Adresse aus der Adresszeile in das Lernprotokoll. Vergleiche deinen Namen für 3 mit der Auflösung.

→ [Auflösung 4](#) am Ende des Kapitels

Break-it 2: Den Ordner assets verschieben

Verschiebe den Ordner assets in den Ordner pages.

- **Vorhersage:** Welche der drei Seiten verlieren ihr Stylesheet? Welche verlieren ihr Logo? Leite die Antwort für jede Seite aus den Regeln für relative Pfade und aus deiner Beobachtung ab.
- **Prüfen:** Lade alle drei Seiten neu.

→ [Auflösung 5](#) am Ende des Kapitels

Predict 3: Das Projekt kopieren

Achtung: Ziehen mit der Maus verschiebt einen Ordner auf demselben Laufwerk, statt ihn zu kopieren.

Wähle den Ordner pfad-training aus und kopiere ihn mit Strg + C (macOS: Cmd + C). Füge ihn auf dem Desktop mit Strg + V (macOS: Cmd + V) ein. Im Finder heißt der Desktop **Schreibtisch**.

- **Vorhersage:**
 1. Zeigt die Startseite der Kopie Logo und Stylesheet?
 2. Welche Adresse hat style.css in der Kopie?
- **Prüfen:** Öffne index.html der Kopie in Chrome. Ersetze in der Adresszeile index.html durch assets/css/style.css und drücke Enter.

→ [Auflösung 6](#) am Ende des Kapitels

5. Prüfen

- ☐ Der Benutzerordner zeigt den Ordner AppData bzw. Library, und learning-notizen.md erscheint mit Endung.
- ☐ Die Adressleiste bzw. Pfadleiste zeigt für projekte einen Pfad wie C:\Users\anna\projekte bzw. /Users/anna/projekte, ohne OneDrive, Desktop oder Dokumente.
- ☐ projekte/toolbox/learning-notizen.md trägt genau diesen Namen.
- ☐ projekte/pfad-training/index.html liegt direkt im Ordner pfad-training.
- ☐ pfad-training ist wie nach dem Entpacken: pages/about.html heißt wieder so, assets liegt direkt in pfad-training, die Kopie aus Predict 3 ist gelöscht.
- ☐ learning-notizen.md enthält zur Beobachtung das Ergebnis je Seite und die Erklärung für **Über**, zu jedem Experiment Vorhersage, Ergebnis und Erklärung.

Zielstand der Toolbox:

```
toolbox/  
└─ learning-notizen.md
```

Quellen

- [MDN – Dealing with files](#)
- [MDN – What is a URL?](#)
- [MDN – Percent-encoding](#)

Auflösungen

Auflösung 1

Zu Beobachtung: Die drei Seiten

Seite	Abweichung
Start	keine
Über	Oben links steht statt des Logos das Symbol für ein fehlendes Bild mit dem Text „Pfad-Training“.
Kontakt	Das Logo ist da, die Gestaltung fehlt: Schrift mit Serifen, weißer Hintergrund, die Links stehen unterstrichen als Liste untereinander.

Auflösung 2

Zu Predict 1: Die Adresszeile

Die Adresse beginnt mit `file:///`. Danach folgt der **absolute Pfad** der Datei, also der Weg vom obersten Ordner aus: unter Windows vom Laufwerk `C:`, unter macOS vom `/` am Anfang. Beispiele sind `file:///C:/Users/anna/projekte/pfad-training/index.html` und `file:///Users/anna/projekte/pfad-training/index.html`.

URLs verwenden auf jedem Betriebssystem den Schrägstrich `/`. Pfade in HTML und CSS stehen deshalb immer mit `/`, auch wenn der Explorer `\` anzeigt.

Auflösung 3

Zu Predict 2: Relative Pfade

#	Pfad	Weg
1	<code>assets/css/style.css</code>	in assets, dann in css
2	<code>../assets/img/logo.svg</code>	aus pages heraus, dann in assets/img
3	<code>../../index.html</code>	aus team heraus, aus pages heraus
4	<code>../about.html</code>	aus team heraus
5	<code>../img/logo.svg</code>	aus css heraus, dann in img

Zeile 2 erklärt das fehlende Logo auf der Seite **Über**. In `about.html` steht `assets/img/logo.svg`. Der Browser sucht deshalb `pages/assets/img/logo.svg`. Diese Datei existiert nicht. Das Kapitel VS Code (Kapitel 1.2) korrigiert den Pfad.

Zu Zeile 5: Pfade in einer CSS-Datei gelten relativ zur CSS-Datei, nicht relativ zur HTML-Seite, die das Stylesheet einbindet.

Der Fehler auf der Seite **Kontakt** hat eine andere Ursache. Das Kapitel Browser & DevTools lokalisiert ihn.

Auflösung 4

Zu Break-it 1: Eine Datei umbenennen

1. Der Browser zeigt eine Fehlerseite (ERR_FILE_NOT_FOUND). Der Link in `index.html` verweist weiterhin auf `pages/about.html`. Wer eine Datei umbenennt, muss alle Pfade anpassen, die auf diese Datei verweisen.
2. Die Adresse kodiert das Leerzeichen als `%20`. Die kopierte Adresse kodiert zusätzlich das ü: `%C3%BCber%20uns.html`. Die Namensregeln vermeiden diese Kodierung.
3. `anfahrt-buero.html`: nur Kleinbuchstaben, ue statt ü, Bindestrich statt Leerzeichen.

Benenne die Datei wieder in `about.html` um.

Auflösung 5

Zu Break-it 2: Den Ordner assets verschieben

Start und **Über** verlieren ihr Stylesheet, weil ihre Pfade auf den alten Ort verweisen. **Kontakt** hatte schon vorher keines. **Start** und **Kontakt** verlieren ihr Logo. Auf der Seite **Über** erscheint dagegen das Logo. Der Pfad `assets/img/logo.svg` führt von `pages/about.html` aus nach `pages/assets/img/`. Dort liegt die Datei jetzt.

Ein Pfad ist nicht für sich genommen richtig oder falsch. Er passt zur Lage der Dateien oder nicht.

Verschiebe `assets` zurück nach `pfad-training`.

Auflösung 6

Zu Predict 3: Das Projekt kopieren

1. Ja. Relative Pfade beschreiben nur den Weg innerhalb des Projekts. Der Ort des Projektordners spielt keine Rolle, solange die Dateien zueinander gleich liegen. Das gilt auch für einen Webserver und für die Rechner anderer Personen.
2. Die Adresse enthält den absoluten Pfad der Kopie, zum Beispiel `file:///C:/Users/anna/Desktop/pfad-training/assets/css/style.css` oder `file:///Users/anna/Desktop/pfad-training/assets/css/style.css`. Chrome zeigt den Inhalt des Stylesheets als Text. In `index.html` steht dafür weiterhin der relative Pfad `assets/css/style.css`.

Ein absoluter Pfad wie `C:\Users\anna\projekte\...` gilt nur auf einem Rechner. Websites verwenden deshalb für eigene Dateien relative Pfade.

Lösche den Ordner `pfad-training` auf dem Desktop.



1.2 VS Code

Voraussetzungen: Dateisystem & Pfade (Kapitel 1.1)

Lernziele

- Ein Projekt als Ordner (Workspace) öffnen und Explorer, Command Palette und Terminal verwenden.
- Relative Pfade mit der Pfad-Vervollständigung schreiben und einen fehlerhaften Pfad korrigieren.
- Den gespeicherten Stand einer Datei vom ungespeicherten Stand im Editor unterscheiden.

1. Vorschau

Zielstand nach diesem Kapitel: Die Toolbox ist als Workspace geöffnet. Links steht der Explorer, daneben das Lernprotokoll mit **Markdown Preview**.

2. Warum

VS Code ist ein kostenloser Code-Editor für Windows, macOS und Linux und in der Webentwicklung weit verbreitet. In einem einfachen Texteditor fällt ein falscher Pfad wie der zum Logo auf der Seite **Über** erst im Browser auf. Die Arbeitseinheit in VS Code ist das Projekt: ein Ordner, dessen Dateien zusammengehören. Alle folgenden Kapitel setzen VS Code als Editor und als Terminal voraus.

Weiterführend: [VS Code – User interface](#) · [VS Code – Workspaces](#)

3. Einrichten

3.1 Installieren

Lade VS Code von code.visualstudio.com und installiere es mit den Voreinstellungen.

Unter macOS ziehst du VS Code aus **Downloads** in den Ordner **Programme** und startest es von dort. Bestätige die Rückfrage von macOS mit **Öffnen**.

Dieses Skript benötigt keine Extensions. VS Code unterstützt HTML, CSS und Markdown ohne Extension.

3.2 Projekt öffnen

Öffne den Ordner `projekte/toolbox` mit **File → Open Folder....** Zeigt VS Code oben den Hinweis **Restricted Mode**, klicke auf **Manage** und im folgenden Tab auf **Trust**.

Öffne immer den Projektordner und nicht eine einzelne Datei. Nur dann zeigt der Explorer alle Dateien und Ordner des Projekts. Ein geöffneter Ordner heißt in VS Code **Workspace**.

Die **Activity Bar** am linken Rand schaltet zwischen den Views um. Ihre Symbole zeigen den Namen beim Überfahren mit der Maus:

View	Zweck
Explorer	Dateien und Ordner des Projekts
Search	Suchen und Ersetzen in allen Dateien des Projekts
Source Control	Git, siehe Git & GitHub (Kapitel 1.4)

Ein Rechtsklick im **Explorer** öffnet ein Kontextmenü. **New File...** und **New Folder...** legen dort eine Datei bzw. einen Ordner an. **Copy** und **Paste** kopieren eine Datei, **Rename...** benennt sie um.

Die **Command Palette** führt jeden Befehl von VS Code über seinen Namen aus: Strg + Shift + P (macOS: Cmd + Shift + P). Hat ein Befehl ein Tastenkürzel, zeigt sie es neben dem Namen.

3.3 Lernprotokoll mit Markdown Preview öffnen

1. Öffne `learning-notizen.md` im **Explorer** von VS Code.
2. Öffne die Command Palette und tippe `preview to the side`. Wähle mit den Pfeiltasten den Eintrag `Markdown: Open Preview to the Side` und bestätige mit Enter.

Rechts neben der Datei öffnet sich die **Markdown Preview**. Sie zeigt „Learning-Notizen“ als Überschrift ohne das Zeichen #. Du schreibst links in der Datei; die Preview zeigt jede Änderung sofort. Führe das Lernprotokoll ab jetzt in VS Code.

3.4 Pfad mit der Pfad-Vervollständigung korrigieren

File → Open Folder... ersetzt das geöffnete Projekt im selben Fenster. **File → Open Recent** wechselt zurück zu den zuletzt geöffneten Projekten. **File → Open File...** (macOS: **File → Open...**) öffnet eine Datei außerhalb des Projektordners.

1. Öffne den Ordner `projekte/pfad-training` mit **File → Open Folder....**
2. Öffne `pages/about.html`.
3. Öffne die Suche mit Strg + F (macOS: Cmd + F) und gib `logo.svg` ein. VS Code markiert den Treffer im Element ``.
4. Schließe die Suche mit Esc. Der Cursor steht wieder im Text.
5. Lösche den Wert des Attributs `src` zwischen den Anführungszeichen und gib ihn neu ein. Die Anführungszeichen bleiben. Beginne mit `../`. Der neue Wert ist `../assets/img/logo.svg` (Zeile 2 in [Predict 2](#) (Kapitel 1.1)).
6. Nach `../` und nach jedem weiteren `/` zeigt VS Code eine Vorschlagsliste. Wähle darin den nächsten Ordner bzw. die Datei mit den Pfeiltasten und Enter. Erscheint keine Liste, lösche das letzte `/` und tippe es erneut.
7. Speichere die Datei (Strg + S, macOS: Cmd + S).
8. Öffne die Seite **Über** in Chrome wie in [Datei im Browser öffnen](#) (Kapitel 1.1) oder lade sie neu, wenn sie offen ist.

Die Seite zeigt das Logo. Die Vorschlagsliste enthält nur Ordner und Dateien, die an der jeweiligen Stelle existieren. Fehlt ein erwarteter Eintrag, liegt die Datei an einem anderen Ort.

3.5 Suchen und Ersetzen

In der Datei: Strg + H (macOS: Cmd + Option + F) öffnet die Felder **Find** und **Replace**. **Replace All** ersetzt alle Treffer der Datei.

In allen Dateien: Die View **Search** durchsucht alle Dateien des Projekts. Die Namen der Symbole erscheinen beim Überfahren mit der Maus:

- **Toggle Replace** blendet das Feld **Replace** ein.
- **Toggle Search Details** zeigt die Felder **files to include** und **files to exclude**. Sie bleiben nach einer Suche gefüllt und gelten für die nächste.
- **Match Case** unterscheidet Groß- und Kleinschreibung.
- **Use Regular Expression** liest die Suche als regulären Ausdruck: ein Suchmuster, in dem Zeichen wie ^ (Zeilenanfang) eine Bedeutung haben.

Über den Treffern steht ihre Anzahl, z. B. 4 results in 2 files. **Replace All** ersetzt nach der Bestätigung mit **Replace** alle Treffer und speichert die geänderten Dateien.

3.6 Terminal

View → Terminal öffnet das integrierte Terminal. Es führt Befehle aus, die du als Text eingibst und mit Enter abschickst. Die letzte Zeile endet mit > (PowerShell) bzw. % (zsh). Dahinter tippst du den Befehl. Unter Windows führt die PowerShell die Befehle aus, unter macOS die zsh. Das Kapitel [Git & GitHub](#) (Kapitel 1.4) verwendet dieses Terminal.

3.7 Tastenkürzel

Aktion	Windows	macOS
Command Palette	Strg + Shift + P	Cmd + Shift + P
Speichern	Strg + S	Cmd + S
In der Datei suchen	Strg + F	Cmd + F
In der Datei ersetzen	Strg + H	Cmd + Option + F
Rückgängig	Strg + Z	Cmd + Z

Die Tabelle nennt die Standardbelegung. Das gültige Kürzel für dein Tastaturlayout steht neben dem Menüeintrag und in der Command Palette.

4. Verstehen

Das Pfad-Training bleibt als Workspace geöffnet. Öffne das Lernprotokoll `projekte/toolbox/learning-notizen.md` mit **File → Open File...** (macOS: **File → Open...**). Die Datei erscheint als Tab, nicht im Explorer. Lege darin die Überschrift `## VS Code (Datum)` an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren. Nimm jede Änderung nach einem Experiment zurück, bevor das nächste beginnt.

Break-it 1: Ungespeicherte Änderung

Ändere im Pfad-Training in `pages/about.html` den Text der Überschrift `<h1>`. Speichere die Datei nicht.

- **Vorhersage:** Welche Überschrift zeigt der Browser? Woran ist in VS Code erkennbar, dass die Datei nicht gespeichert ist?
- **Prüfen:** Lade die Seite neu. Betrachte in VS Code den Tab der Datei und das Symbol des Explorers.

→ [Auflösung 1](#) am Ende des Kapitels

Predict 1: Ordner des Terminals

Das Pfad-Training ist als Workspace geöffnet. Öffne das Terminal. Der Befehl `pwd` gibt den Ordner aus, in dem das Terminal arbeitet.

- **Vorhersage:** Welchen Pfad gibt der Befehl `pwd` aus? Welchen Pfad gibt er aus, nachdem du die Toolbox geöffnet und dort ein Terminal gestartet hast?
- **Prüfen:** Führe `pwd` in beiden Projekten aus. Lass danach die Toolbox geöffnet.

→ [Auflösung 2](#) am Ende des Kapitels

5. Prüfen

- ☐ VS Code öffnet die Toolbox als Ordner. Der Explorer zeigt `learning-notizen.md`. Neben der Datei steht ihre **Markdown Preview**. Fehlt die Preview, öffne sie wie in [Lernprotokoll mit Markdown Preview öffnen](#).
- ☐ Die Seite **Über** im Pfad-Training zeigt das Logo.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.

Quellen

- [VS Code – User interface](#)
- [VS Code – What is a VS Code workspace?](#)
- [VS Code – User and workspace settings](#)
- [VS Code – Terminal Basics](#)
- [VS Code – Keyboard shortcuts for Windows \(PDF\)](#) · [für macOS \(PDF\)](#)

Auflösungen

Auflösung 1

Zu Break-it 1: Ungespeicherte Änderung

Der Browser zeigt die alte Überschrift. Er liest die Datei aus dem Dateisystem, und dort steht der zuletzt gespeicherte Stand. VS Code kennzeichnet eine ungespeicherte Datei mit einem Punkt im Tab. Das Symbol des Explorers zeigt die Anzahl der ungespeicherten Dateien.

Zeigt der Browser eine Änderung nicht an, prüfe zuerst, ob die Datei gespeichert ist.

Speichere die Datei und lade die Seite neu. Klicke danach in den Tab `about.html` und drücke Strg + Z (macOS: Cmd + Z), bis die Überschrift wieder „Über das Pfad-Training“ lautet. Speichere erneut.

Auflösung 2

Zu Predict 1: Ordner des Terminals

Das Terminal startet im Ordner des Workspace. Die Ausgabe ist der absolute Pfad von `pfad-training` bzw. von `toolbox`, zum Beispiel `C:\Users\<name>\projekte\pfad-training` (PowerShell zeigt ihn unter der Überschrift `Path`) oder `/Users/<name>/projekte/pfad-training`.

Befehle im Terminal beziehen sich auf diesen Ordner. Ein Git-Befehl im Terminal der Toolbox wirkt deshalb auf die Toolbox und nicht auf ein anderes Projekt.



1.3 Browser & DevTools

Voraussetzungen: [Dateisystem & Pfade](#) (Kapitel 1.1), [VS Code](#) (Kapitel 1.2)

Lernziele

- DOM, Styles und geladene Ressourcen einer Seite mit den Chrome DevTools untersuchen.
- Eine fehlende Ressource über Console und Network lokalisieren und im Editor korrigieren.
- Eine Seite bei 320 px Breite und im Dark Mode prüfen und Rolle und Name eines Elements ablesen.

1. Vorschau

Die DevTools von Chrome: links die Seite, rechts das Panel **Elements** mit dem [DOM](#) und den Styles des ausgewählten Elements.

2. Warum

Der Browser zeigt das Ergebnis einer Seite, aber nicht dessen Zustandekommen. Fehlt ein Bild oder ein [Stylesheet](#), zeigt die Seite keine Fehlermeldung. Die DevTools zeigen, welche Dateien der Browser geladen hat, wie er sie interpretiert und welches Ergebnis daraus entsteht. Sie sind die erste Anlaufstelle, wenn eine Seite vom erwarteten Ergebnis abweicht.

Weiterführend: [MDN – What are browser developer tools?](#) · [Chrome DevTools – Overview](#)

3. Einrichten

Die Tastenkürzel stehen in der Tabelle [Tastenkürzel](#).

3.1 Chrome als Referenzbrowser

Chrome ist seit [Dateisystem & Pfade](#) (Kapitel 1.1) installiert. Dieses Skript verwendet Chrome als Referenzbrowser. Edge, Firefox und Safari enthalten vergleichbare Werkzeuge mit abweichender Oberfläche.

Ausgangsstand ist das Pfad-Training nach dem Kapitel [VS Code](#) (Kapitel 1.2): Die Seite **Über** zeigt das Logo, die Seite **Kontakt** erscheint ohne Gestaltung.

Öffne `projekte/pfad-training/index.html` in Chrome.

3.2 DevTools öffnen

Öffne die DevTools. Sie erscheinen im Fenster von Chrome neben oder unter der Seite. Erscheint ein Hinweis zur Sprache, klicke auf **Don't show again**.

Die DevTools gliedern sich in **Panels: Elements, Console, Sources, Network** und weitere. Panels ohne Platz in der Leiste stehen hinter ».

3.3 Command Menu

Das **Command Menu** führt jeden Befehl der DevTools über seinen Namen aus. Der Fokus muss dabei in den DevTools liegen. Ein Klick in die DevTools setzt den Fokus dorthin.

Öffne das Command Menu, gib `Show Console` ein und drücke Enter. Die DevTools zeigen das Panel **Console**.

3.4 Elements

1. Schalte auf der Startseite den **Inspect mode** ein und klicke auf die Überschrift **Pfad-Training**.
2. Das Panel **Elements** markiert das Element `<h1>`. Doppelklicke im Panel **Elements** auf den Text `Pfad-Training` zwischen `<h1>` und `</h1>`, ändere ihn und drücke Enter. Die Seite zeigt den neuen Text.
3. Der Tab **Styles** zeigt alle CSS-Regeln des Elements, darunter zwei Regeln `h1`. Neben der einen steht `style.css:...`; die Angabe nennt Datei und Zeile. Die andere stammt aus dem **user agent stylesheet**, den Vorgaben des Browsers.
4. Klicke in der Regel `h1` aus `style.css` auf den Wert `var(--accent)` von `color`, tippe `green` und drücke Enter. Die Überschrift wird grün.
5. Klappe mit dem Dreieck vor `<header>` das Element auf und klicke auf die Zeile `<nav>`. Das Panel markiert `<nav>`.

Zum Nachschlagen in späteren Kapiteln:

- Im **Inspect mode** wählt ein Klick auf einen Link den Link nur aus. Der Browser öffnet ihn nicht.
- Nach einem Befehl im **Command Menu** ist der **Inspect mode** aus.
- Das Dreieck vor einer Zeile klappt ein Element auf. Das umgebende Element steht darüber mit geringerer Einrückung.
- Ein Klick auf eine Zeile wählt das Element aus, auch ein Element ohne sichtbare Fläche.
- Beim Überfahren einer Zeile hebt Chrome das Element hervor. Ein Tooltip nennt Tag und Größe.
- Das Badge `flex` neben `<header>` kennzeichnet ein Element mit `display: flex`.
- Im **Inspect mode** zeigt der Tooltip über Text unter **Accessibility** die Zeile **Contrast**.
- Die Regel **element.style** im Tab **Styles** gilt nur für das ausgewählte Element. Ein Klick zwischen ihre Klammern legt eine Deklaration an; der Doppelpunkt springt zum Wert.
- Ein Klick auf das Farbfeld vor einem Farbwert öffnet den **Color Picker**. Bei Textfarben zeigt er die **Contrast ratio**.
- Der Tab **Computed** neben **Styles** zeigt die berechneten Werte. Das Feld **Filter** sucht eine Eigenschaft.
- Der Tab **Layout** blendet Overlays für Grid und Flexbox ein.

3.5 Network

Die DevTools zeichnen Anfragen nur auf, solange sie geöffnet sind. Wurde die Seite vor dem Öffnen der DevTools geladen, füllt sich die Liste im Panel **Network** erst beim nächsten Laden. Jede Zeile der Liste ist eine Anfrage: eine Datei, die der Browser angefordert hat. Ein Klick auf eine Zeile zeigt im Tab **Headers** die **Request URL**.

3.6 Device Toolbar

Schalte die **Device Toolbar** ein und wähle unter **Dimensions** den Eintrag **Responsive**. Die Seite erscheint in der Breite, die das erste Feld nennt. Die zwei Felder daneben setzen Breite und Höhe des Viewports. Unter **Dimensions** stehen außerdem Geräte mit fester Breite und Höhe. Schalte die **Device Toolbar** danach wieder aus.

3.7 Accessibility

Ein **Screenreader** liest Menschen mit Sehbehinderung eine Seite vor. Er verwendet dafür nicht die sichtbare Darstellung, sondern den Accessibility Tree. Der Accessibility Tree enthält zu jedem Element eine **Rolle** (Art des Elements) und einen **Namen** (Bezeichnung des Elements).

Der Tab **Accessibility** im Panel **Elements** steht neben **Styles**, bei schmalen Panel hinter ». Er zeigt unter **Computed properties** die Werte **Name** und **Role** des ausgewählten Elements.

3.8 Kurzreferenz

Werkzeug	Zeigt	Einsatz
Elements mit Styles	DOM und CSS-Regeln eines Elements	Die Darstellung weicht vom erwarteten Ergebnis ab.
Console	Fehlermeldungen	Ein Teil der Seite fehlt oder funktioniert nicht.
Network	angeforderte Dateien und ihre Pfade	Ein Bild, ein Stylesheet oder eine Schrift lädt nicht.
Device Toolbar	die Seite in anderen Viewport-Größen	Prüfung der Darstellung auf kleinen Viewports
Accessibility	Rolle und Name eines Elements	Prüfung der Barrierefreiheit
Rendering (Command Menu: Show Rendering)	<u>Color Scheme</u> und weitere Einstellungen des Betriebssystems, die die DevTools emulieren	Prüfung von Light Mode und Dark Mode

3.9 Tastenkürzel

Aktion	Windows	macOS
Datei in Chrome öffnen	Strg + O	Cmd + O
Seite neu laden	F5	Cmd + R
DevTools öffnen	F12 oder Strg + Shift + I	Cmd + Option + I
Inspect mode ein- und ausschalten	Strg + Shift + C	Cmd + Shift + C
Command Menu	Strg + Shift + P	Cmd + Shift + P

Aktion	Windows	macOS
Device Toolbar ein- und ausschalten	Strg + Shift + M	Cmd + Shift + M

Die Kürzel für **Command Menu** und **Device Toolbar** wirken nur mit Fokus in den DevTools.

4. Verstehen

Lege im Lernprotokoll die Überschrift ## Browser & DevTools (Datum) an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren. Nimm jede Änderung nach einem Experiment zurück, bevor das nächste beginnt.

Break-it 1: Seite neu laden

- **Vorhersage:** In Elements hast du Text und Farbe der Überschrift geändert. Welchen Text und welche Farbe zeigt sie nach dem Neuladen der Seite?
- **Prüfen:** Lade die Seite neu.

→ Auflösung 1 am Ende des Kapitels

Predict 1: Ursache des Fehlers

- **Vorhersage:** Die Seite **Kontakt** erscheint ohne Gestaltung. Welche Ursache hat die fehlende Gestaltung?
- **Prüfen:** Öffne die Seite **Kontakt** und das Panel **Console**.

→ Auflösung 2 am Ende des Kapitels

Predict 2: Anfragen der Seite

- **Vorhersage:** Wie viele Dateien fordert die Seite **Kontakt** an? Welche Anfrage schlägt fehl?
- **Prüfen:** Öffne das Panel **Network** und lade die Seite neu. Wähle die Anfrage mit dem Status (failed) aus und lies die **Request URL**.

→ Auflösung 3 am Ende des Kapitels

Korrigiere den Pfad in VS Code:

1. Öffne in VS Code mit **File → Open File...** (macOS: **File → Open...**) die Datei pfad-training/pages/team/kontakt.html.
2. Ändere in Zeile 7 im Element <Link> den Pfad ../../assets/css/styles.css in ../../assets/css/style.css.
3. Speichere die Datei und lade die Seite neu.

Das Panel **Network** zeigt danach keine fehlgeschlagene Anfrage.

Predict 3: Darstellung bei 320 px

- **Vorhersage:** 320 px entspricht der Breite eines kleinen Smartphones. Welches Element der Startseite passt nicht in diese Breite?
- **Prüfen:** Öffne die Startseite, schalte die **Device Toolbar** ein und setze die Breite auf 320. Scrolle nach rechts. Erscheint der Mauszeiger als Kreis, bildet die Device Toolbar einen Touchscreen nach; ziehe die Seite dann mit gedrückter Maustaste nach links.

→ [Auflösung 4](#) am Ende des Kapitels

Predict 4: Rolle und Name des Logos

- **Vorhersage:** Welche Rolle und welchen Namen hat das Logo der Startseite? Welche Rolle hat der Bereich mit den drei Links?
- **Prüfen:** Wähle das Logo mit dem **Inspect mode** aus und lies im Tab **Accessibility Name** und **Role**. Klicke danach im Panel **Elements** auf die Zeile `<nav>` direkt unter dem ausgewählten `<img>` und vergleiche.

→ [Auflösung 5](#) am Ende des Kapitels

Predict 5: Color Scheme

- **Vorhersage:** Die Seite enthält kein Bedienelement für einen Dark Mode. Wie erscheint sie bei hellem, wie bei dunklem Betriebssystem?
- **Prüfen:** Führe im Command Menu nacheinander die Befehle **Emulate CSS prefers-color-scheme: light** und **Emulate CSS prefers-color-scheme: dark** aus. Der Befehl **Do not emulate CSS prefers-color-scheme** beendet die Emulation.

→ [Auflösung 6](#) am Ende des Kapitels

5. Prüfen

- ☐ Alle drei Seiten des Pfad-Trainings zeigen Logo und Stylesheet. Die Korrektur steht in der Datei, nicht nur in den DevTools.
- ☐ Das Panel **Network** zeigt für keine der drei Seiten eine fehlgeschlagene Anfrage.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.

Quellen

- [MDN – What are browser developer tools?](#)
- [MDN – Accessibility tree](#)
- [MDN – prefers-color-scheme](#)
- [Chrome DevTools – Overview](#)
- [Chrome DevTools – Simulate mobile devices with device mode](#)
- [Chrome DevTools – Accessibility features reference](#)
- [Chrome DevTools – Emulate CSS media features](#)

Auflösungen

Auflösung 1

Zu [Break-it 1: Seite neu laden](#)

Die Überschrift zeigt wieder den ursprünglichen Text und die ursprüngliche Farbe. Die DevTools ändern den **DOM**, also den Zustand der Seite im Arbeitsspeicher des Browsers. Die Dateien bleiben unverändert.

Die DevTools eignen sich deshalb zum Ausprobieren. Dauerhafte Änderungen entstehen im Editor.

Auflösung 2

Zu [Predict 1: Ursache des Fehlers](#)

Die Console zeigt den Fehler `Failed to load resource: net::ERR_FILE_NOT_FOUND`, rechts daneben den Namen der Datei `styles.css`. Der Browser hat eine Datei angefordert, die nicht existiert. Unter welchem Pfad er sie gesucht hat, zeigt das Panel **Network** in Predict 2.

Die Console sammelt die Fehlermeldungen einer Seite. Die Seite selbst zeigt diese Fehler nicht an.

Auflösung 3

Zu [Predict 2: Anfragen der Seite](#)

Die Seite fordert drei Dateien an: `kontakt.html`, `styles.css` und `logo.svg`. Die Anfrage für `styles.css` schlägt fehl. Die Request URL nennt den angeforderten Pfad `.../pfad-training/assets/css/styles.css`. Die Datei im Ordner heißt `style.css`.

Ein Webserver beantwortet eine solche Anfrage mit dem Status **404 Not Found**. Bei lokalen Dateien gibt es keinen Server. Der Browser meldet stattdessen `ERR_FILE_NOT_FOUND`.

Auflösung 4

Zu [Predict 3: Darstellung bei 320 px](#)

Der Ordnerbaum passt nicht in die Breite. Er steht in einem Element `<pre>`, das Zeilen nicht umbricht. Dadurch wird die Seite breiter als der Viewport und lässt sich horizontal scrollen.

Horizontales Scrollen ist ein häufiger Fehler auf kleinen Viewports. Die Kapitel `html-css` prüfen jede Seite, deren Layout sie bauen, bei 320 px.

Auflösung 5

Zu [Predict 4: Rolle und Name des Logos](#)

Das Logo hat die Rolle `image` und den Namen `Pfad-Training`. Der Name stammt aus dem Attribut `alt`. Ohne `alt` hat das Bild keinen Namen, und ein Screenreader kann seinen Inhalt nicht wiedergeben.

Das Element `<nav>` hat die Rolle `navigation`. Ein Screenreader kann diesen Bereich direkt ansteuern. Das Kapitel `Das Skelett` behandelt die Rollen der Seitenbereiche.

Auflösung 6

Zu [Predict 5: Color Scheme](#)

Mit **light** erscheint die Seite hell, mit **dark** dunkel. Das Stylesheet enthält Regeln für beide Color Schemes. Der Browser wählt das Color Scheme nach der Einstellung des Betriebssystems. Die DevTools emulieren diese Einstellung, ohne das Betriebssystem umzustellen.

Dieselbe Option steht im Tab **Rendering**. Ihn öffnet der Befehl **Show Rendering** im Command Menu.

1.4 Git & GitHub

Voraussetzungen: Dateisystem & Pfade (Kapitel 1.1), VS Code (Kapitel 1.2)

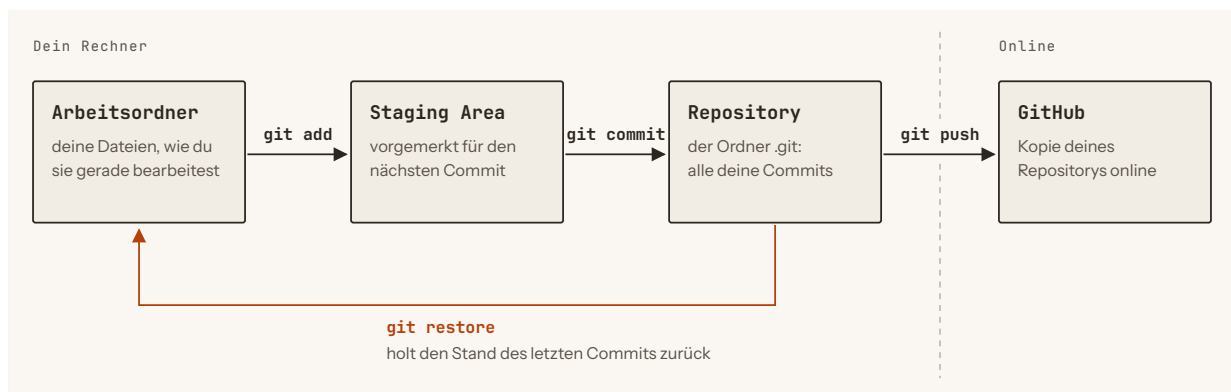
Lernziele

- Ein lokales Repository anlegen und Änderungen über Staging Area und Commit versionieren, im Terminal und in VS Code.
- Commit-Nachrichten nach Conventional Commits formulieren.
- Ein Repository auf GitHub veröffentlichen und den Stand des letzten Commits mit `git restore` wiederherstellen.

1. Vorschau

Zielstand nach diesem Kapitel: Die Toolbox liegt als privates Repository mit drei Commits auf GitHub.

Das Modell hinter den Befehlen dieses Kapitels:



2. Warum

Git speichert Stände eines Projekts als Commits. Jeder Commit lässt sich wiederherstellen und mit anderen Ständen vergleichen. **GitHub** ist eine Plattform, auf der eine Kopie des Repositorys liegt: als Sicherung, zum Teilen und für die Abgabe. Git ist der Standard der Versionskontrolle in der Softwareentwicklung.

Weiterführend: MDN – Version control · Pro Git, Kapitel 1 und 2

3. Einrichten

Ab Repository anlegen nennt dieses Kapitel jeden Git-Schritt zweimal: als Befehl im **Terminal** und als Aktion in **VS Code**. Beide Wege arbeiten auf demselben Repository. Das Terminal zeigt die Antworten von Git im Wortlaut. VS Code stellt denselben Zustand grafisch dar. Führe jeden Schritt auf einem der beiden Wege aus und betrachte das Ergebnis auf dem anderen. Wechsle zwischen den Wegen, zum Beispiel den ersten Commit im Terminal und den zweiten in VS Code.

Das Terminal ist das integrierte Terminal von VS Code (**View → Terminal**).

3.1 Git installieren

Windows	macOS
Lade Git von git-scm.com und starte die Installation. Wähle auf der Seite Choosing the default editor used by Git in der Liste Use Visual Studio Code as Git's default editor . Bestätige alle anderen Seiten mit Next .	Führe im Terminal <code>git --version</code> aus. Fehlt Git, bietet macOS die Installation der Befehlszeilen-Entwicklertools an. Wähle Installieren und danach Akzeptieren . Warte auf die Meldung „Die Software wurde installiert.“ und wähle Fertig .

Starte VS Code neu und prüfe die Installation im Terminal:

```
git --version
```

Git antwortet mit `git version` und der Versionsnummer.

3.2 GitHub-Konto anlegen

1. Lege auf github.com ein Konto an. Der Benutzername ist Teil der Adresse jedes Repositorys.
2. Wähle oben rechts dein Profilbild → **Settings** → **Emails**. Aktiviere **Keep my email addresses private**.
3. Kopiere die Ersatzadresse aus dem Text unter der Option, zum Beispiel `12345678+anna-muster@users.noreply.github.com`.

Jeder Commit enthält eine E-Mail-Adresse. In einem öffentlichen Repository ist sie für alle lesbar. Die Ersatzadresse verhindert, dass die private Adresse veröffentlicht wird.

3.3 Git konfigurieren

Führe die folgenden Befehle einmal pro Rechner aus. Setze deinen Namen und die Ersatzadresse ein. Der Name steht in jedem Commit und ist für alle sichtbar, die das Repository öffnen können.

```
git config --global user.name "Anna Muster"
git config --global user.email "12345678+anna-muster@users.noreply.github.com"
git config --global init.defaultBranch main
```

Prüfe die Konfiguration:

```
git config --global --list
```

Die Ausgabe enthält `user.name=Anna Muster`, `user.email=12345678+anna-muster@users.noreply.github.com` und `init.defaultbranch=main`. Git schreibt die Namen der Einstellungen klein.

Die Option `--global` setzt die Werte für alle Repositories auf dem Rechner. `init.defaultBranch main` gibt dem ersten Branch jedes neuen Repositorys den Namen `main`.

3.4 Repository anlegen

Öffne die Toolbox in VS Code. Prüfe mit `pwd` im Terminal, dass der Pfad auf `toolbox` endet.

Terminal	VS Code
----------	---------

<code>git init</code>	Source Control → Initialize Repository
-----------------------	--

Git antwortet mit `Initialized empty Git repository in .../toolbox/.git/` und legt im Projekt den Ordner `.git` an. Er enthält die gesamte Historie des Projekts.

Achtung: Ändere oder lösche den Ordner `.git` nicht von Hand. Mit dem Ordner geht die Historie verloren.

VS Code blendet `.git` im Explorer aus. Der Dateimanager zeigt den Ordner, wenn versteckte Dateien eingeblendet sind (siehe Dateisystem & Pfade (Kapitel 1.1)).

3.5 Zustand abfragen: `git status`

Terminal	VS Code
----------	---------

<code>git status</code>	Source Control zeigt die Liste Changes
-------------------------	--

Ausgabe im Terminal:

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  learning-notizen.md

nothing added to commit but untracked files present (use "git add" to track)
```

Untracked bedeutet: Die Datei liegt im Arbeitsordner, aber Git versioniert sie noch nicht. VS Code kennzeichnet solche Dateien mit **U**.

`git status` nennt in den eingeklammerten Zeilen die Befehle, die im aktuellen Zustand möglich sind.

Hast du den Ordner `toolbox` im Finder geöffnet, nennt `git status` unter macOS zusätzlich die Datei `.DS_Store` unter `Untracked files`. Der Finder legt sie in jedem Ordner an, den er anzeigt. Sie gehört nicht in das Repository.

3.6 Dateien ausschließen: `.gitignore`

Lege im Explorer von VS Code mit **New File...** die Datei `.gitignore` an und speichere sie mit diesem Inhalt:

```
# Dateien des Betriebssystems
.DS_Store
Thumbs.db
```

Eine Zeile, die mit # beginnt, ist ein Kommentar; Git wertet sie nicht aus. Thumbs.db legt der Windows-Explorer in manchen Ordnern mit Vorschaubildern an.

Führe erneut `git status` aus. Unter macOS fehlt .DS_Store jetzt in der Ausgabe. Neu unter Untracked files ist .gitignore. Die Datei gehört in das Repository, damit die Regeln für alle Beteiligten gelten.

3.7 Commit-Nachrichten: Conventional Commits

Dieses Skript verwendet für Commit-Nachrichten die Konvention [Conventional Commits 1.0.0](#). Eine Nachricht hat die Form:

```
<type>: <description>
```

Die spitzen Klammern markieren Platzhalter und gehören nicht zur Nachricht, zum Beispiel `fix: Pfad zum Logo korrigieren`.

type	Verwendung	Beispiel
feat	neuer Inhalt oder neue Funktion	feat: Navigation ergänzen
fix	Korrektur eines Fehlers	fix: Pfad zum Logo korrigieren
docs	Dokumentation, README, Lernprotokoll	docs: Aufbau der Toolbox beschreiben
refactor	Umbau ohne Änderung von Inhalt oder Darstellung	refactor: Stylesheet in Dateien aufteilen
chore	Projektorganisation	chore: .gitignore hinzufügen

Die Spezifikation definiert `feat` und `fix` und lässt weitere Typen zu, darunter `docs`, `refactor` und `chore`. Die `description` benennt die Änderung in einer Zeile. Ein Commit enthält genau eine thematisch zusammengehörige Änderung.

3.8 Erster Commit

Ein Commit entsteht in zwei Schritten (siehe Abbildung in der [Vorschau](#)):

1. `git add` überträgt Änderungen in die [Staging Area](#). Die Staging Area enthält den Inhalt des nächsten Commits.
2. `git commit` speichert den Inhalt der Staging Area mit einer Nachricht als neuen Stand im **Repository**.

Die zwei Schritte erlauben es, aus mehreren Änderungen gezielt die zusammengehörigen auszuwählen. Der erste Commit enthält nur das Lernprotokoll. Die Schaltflächen neben einer Datei in **Source Control** erscheinen, wenn der Mauszeiger auf der Zeile steht.

Achtung: `git commit` ohne `-m` öffnet einen Editor für die Nachricht. Unter Windows ist das ein Tab in VS Code; das Terminal wartet, bis der Tab geschlossen ist. Unter macOS ist es in der Voreinstellung Vim. Verlasse Vim mit `Esc`, `:q!`, `Enter` und wiederhole den Befehl mit `-m`.

Schritt	Terminal	VS Code
In die Staging Area	<code>git add learning-notizen.md</code>	Source Control → + (Stage Changes) neben der Datei
Prüfen	<code>git status</code> zeigt die Dateien unter Changes to be committed	Die Dateien stehen unter Staged Changes
Commit	<code>git commit -m "docs: Lernprotokoll anlegen"</code>	Nachricht eingeben → Commit

Ausgabe von `git status` nach `git add`:

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   learning-notizen.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
```

Unter Windows kann `git add` melden: warning: in the working copy of '...', LF will be replaced by CRLF the next time Git touches it. Git passt die Zeilenenden an; die Meldung erfordert keine Aktion.

`learning-notizen.md` liegt in der Staging Area, `.gitignore` noch nicht. Git antwortet auf den Commit mit einer Zeile wie `[main (root-commit) 11df2b3] docs: Lernprotokoll anlegen`. Nach dem Commit nennt `git status` die Datei `learning-notizen.md` nicht mehr. Die Datei entspricht dem Stand des letzten Commits.

VS Code kennzeichnet Dateien in **Source Control** mit einem Buchstaben:

Kennzeichnung	git status	Bedeutung
U	unter Untracked files	neue Datei, nicht versioniert
A	new file:	neue Datei in der Staging Area
M	modified:	versionierte Datei, seit dem letzten Commit geändert
D	deleted:	versionierte Datei, gelöscht

3.9 Zweiter Commit

Der zweite Commit enthält die Datei `.gitignore`.

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Changes) über der Liste Changes
Prüfen	<code>git status</code> zeigt die Dateien unter Changes to be committed	Die Dateien stehen unter Staged Changes
Commit	<code>git commit -m "chore: .gitignore hinzufügen"</code>	Nachricht eingeben → Commit

`git add .` überträgt alle Änderungen des aktuellen Ordners in die Staging Area. Die Zeile **Prüfen** zeigt, welche Änderungen der Commit enthält.

3.10 Historie anzeigen

Terminal	VS Code
<code>git log --oneline</code>	Source Control → Bereich Graph

```
7c5e214 (HEAD → main) chore: .gitignore hinzufügen
11df2b3 docs: Lernprotokoll anlegen
```

Jeder Commit hat eine eindeutige Kennung, den **Hash**. Die Ausgabe kürzt ihn auf sieben Zeichen. Die Hashes unterscheiden sich von Repository zu Repository. (HEAD → main) markiert den neuesten Commit des Branch main.

3.11 Auf GitHub veröffentlichen

Achtung: Wähle einen der beiden Wege. **Publish Branch** in VS Code legt das Repository auf GitHub selbst an; die Schritte 1 bis 4 und die Zeile **Remote eintragen** entfallen dann. Beide Wege zusammen erzeugen zwei Repositories.

Lege das Repository auf GitHub an:

1. Wähle auf github.com + → **New repository**.
2. Setze den Namen auf `toolbox` und die Sichtbarkeit auf **Private**.
3. Lass die Optionen für README, `.gitignore` und Lizenz deaktiviert. Das Repository muss leer sein.
4. Wähle **Create repository**.

Verbinde das lokale Repository mit einem Remote und übertrage die Commits. Kopiere die Adresse des Remote oben auf der Seite des neuen Repositories unter **Quick setup**. Sie beginnt mit `https://`, endet auf `.git` und enthält deinen Benutzernamen statt `anna-muster`.

Schritt	Terminal	VS Code
Remote eintragen	<code>git remote add origin https://github.com/anna-muster/toolbox.git</code>	–
Übertragen	<code>git push -u origin main</code>	Source Control → Publish Branch → Publish to GitHub private repository

Der erste Push und **Publish Branch** erfordern eine Anmeldung bei GitHub im Browser. Im Terminal unter Windows öffnet sich dafür ein Fenster mit **Sign in with your browser**. Unter macOS und bei **Publish Branch** fragt VS Code nach der Anmeldung mit „The extension ‘GitHub’ wants to sign in using GitHub.“. Wähle **Allow** und melde dich im Browser an. Der Rechner speichert die Anmeldung.

Lade danach die Seite des Repositories auf GitHub neu. Sie zeigt `.gitignore`, `learning-notizen.md` und **2 Commits**.

Die Option `-u origin main` ist nur beim ersten Mal nötig. Danach genügt `git push` bzw. **Sync Changes**.

4. Verstehen

Lege im Lernprotokoll die Überschrift ## Git & GitHub (Datum) an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren. Nimm jede Änderung nach einem Experiment zurück, bevor das nächste beginnt. Die Notizen sind der Inhalt des dritten Commits.

Predict 1: Eine Änderung ohne Commit

Ergänze in .gitignore die Zeile # Test und speichere die Datei. Erstelle keinen Commit.

- **Vorhersage:** Was geschieht mit der Testzeile, wenn du die Datei mit `git restore` zurücksetzt?
- **Prüfen:** Setze die Datei zurück und öffne sie:

Terminal	VS Code
<code>git restore .gitignore</code>	Source Control → Discard Changes neben der Datei → Discard File

→ [Auflösung 1](#) am Ende des Kapitels

Break-it 1: Eine versionierte Datei löschen

Lösche .gitignore im Explorer von VS Code.

- **Vorhersage:** Kann Git die Datei wiederherstellen?
- **Prüfen:**
 1. Führe `git status` aus bzw. betrachte **Source Control**.
 2. Führe `git restore .gitignore` aus bzw. wähle **Discard Changes** neben der Datei und bestätige die Rückfrage mit **Restore File**. Betrachte den Explorer.

→ [Auflösung 2](#) am Ende des Kapitels

Predict 2: Ein leerer Ordner

Lege in der Toolbox den leeren Ordner entwuerfe an.

- **Vorhersage:** Was zeigt `git status`?
- **Prüfen:** Führe `git status` aus.

→ [Auflösung 3](#) am Ende des Kapitels

Commit

Schließe die Notizen zu diesem Kapitel ab und übertrage sie im dritten Commit. Formuliere die Nachricht selbst nach [Conventional Commits](#).

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Changes) über der Liste Changes
Prüfen	<code>git status</code> zeigt die Dateien unter Changes to be committed	Die Dateien stehen unter Staged Changes
Commit	<code>git commit -m "<type>: <description>"</code>	Nachricht eingeben → Commit

Schritt	Terminal	VS Code
Übertragen	<code>git push</code>	Sync Changes

Öffne danach das Repository auf GitHub und wähle **3 Commits** und danach die Nachricht des letzten Commits. GitHub zeigt die Dateien, die der Commit ändert: nur `learning-notizen.md`.

Jedes folgende Kapitel endet mit einem Commit und einem Push.

5. Prüfen

- ☐ `git status` meldet `nothing to commit, working tree clean` und `Your branch is up to date with 'origin/main'`.
- ☐ `git log --oneline` zeigt drei Commits mit Nachrichten nach Conventional Commits.
- ☐ Das private Repository toolbox auf GitHub zeigt dieselben drei Commits.
- ☐ `git log --format="%h %ae"` zeigt bei allen drei Commits die Ersatzadresse.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.

Zielstand der Toolbox:

```
toolbox/
├── .git/
├── .gitignore
└── learning-notizen.md
```

Quellen

- [Conventional Commits 1.0.0](#)
- [MDN – Version control](#)
- [Pro Git \(Deutsch\) – Scott Chacon, Ben Straub](#)
- [Git – Downloads](#)
- [VS Code – Source Control](#)
- [GitHub Docs – Setting your commit email address](#)
- [GitHub Docs – Ignoring files](#)

Auflösungen

Auflösung 1

Zu Predict 1: Eine Änderung ohne Commit

Die Testzeile ist entfernt, Git kann sie nicht wiederherstellen. `git restore` setzt die Datei auf den Stand des letzten Commits zurück. Änderungen ohne Commit sind Git nicht bekannt. Erstelle deshalb nach jedem funktionierenden Zwischenstand einen Commit.

Auflösung 2

Zu Break-it 1: Eine versionierte Datei löschen

`git status` zeigt `deleted: .gitignore` unter `Changes not staged for commit`, VS Code kennzeichnet die Datei mit **D**. Daneben steht `modified: learning-notizen.md`: Die Notizen zu diesem Kapitel sind noch nicht committet. Lag `.DS_Store` in 3.5 vor, nennt `git status` unter macOS zusätzlich `.DS_Store` unter `Untracked files`: Ohne `.gitignore` gilt die Regel nicht. Das Repository enthält den Stand der Datei aus dem letzten Commit. `git restore` stellt diesen Stand im Arbeitsordner wieder her.

Die Break-it-Aufgaben der folgenden Kapitel setzen voraus, dass der Stand vor dem Experiment committet ist.

Auflösung 3

Zu Predict 2: Ein leerer Ordner

`git status` nennt den Ordner `entwuerfe` nicht. Git versioniert Dateien, keine Ordner. Ein Ordner ist Git nur über die Dateien bekannt, die er enthält. Ein leerer Ordner gelangt deshalb nicht in das Repository und nicht auf GitHub.

Lösche den Ordner `entwuerfe`.



1.5 Markdown & README

Voraussetzungen: VS Code (Kapitel 1.2), Git & GitHub (Kapitel 1.4)

Lernziele

- Begründen, warum ein Projekt Dokumentation braucht und welche Fragen eine README beantwortet.
- Markdown-Quelltext lesen und prüfen: Überschriften, Absätze, Listen, Links, Code und Tabellen.
- Eine README mit Zweck, Aufbau und Fortschritt der Toolbox anlegen und mit einem Commit auf GitHub übertragen.

1. Vorschau

Zielstand nach diesem Kapitel: Die Toolbox enthält die Datei `README.md` mit diesem Inhalt.

```
# Toolbox

Bauteile für das Web: moderne HTML- und CSS-Patterns zum Nachschlagen – gebaut
mit genau diesen Patterns.

## Struktur

...

toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
└── .gitignore          ← Systemdateien, die nicht ins Repository gehören
...

## Fortschritt

| Kapitel | Status |
|---|---|
| Das Skelett | offen |

## Quellen

- [MDN Web Docs](https://developer.mozilla.org/) – Primärquelle für alle Einträge
```

2. Warum

Zweck und Zusammenhang der Dateien eines Projekts kennt zunächst nur die Person, die sie angelegt hat. Eine **README** beschreibt das Projekt für alle anderen: Teammitglieder, Lehrende und dich selbst nach einigen Monaten. GitHub zeigt die README aus dem obersten Ordner des Repositorys auf der Startseite. Geschrieben ist sie in **Markdown**, einem Textformat, das GitHub, GitLab und VS Code ohne Erweiterung darstellen.

Weiterführend: [GitHub Docs – About the repository README file](#) · [CommonMark – Markdown Reference](#)

3. Einrichten

Die Tastenkürzel stehen in der Tabelle [Tastenkürzel](#).

3.1 Was eine README beantwortet

Eine README beantwortet drei Fragen, in dieser Reihenfolge:

Teil	Inhalt	In der Toolbox
Zweck	wozu das Projekt dient, in ein bis zwei Sätzen	Titel und erster Absatz
Start	wie man das Projekt öffnet oder ausführt	Abschnitt ## Ansehen, ab Das Skelett
Aufbau	welche Dateien und Ordner es gibt und wofür	Abschnitt ## Struktur

Die Toolbox enthält noch keine Seite. Der Abschnitt ## Ansehen folgt deshalb im nächsten Kapitel Das Skelett, das die erste HTML-Datei anlegt. Der Abschnitt ## Fortschritt ist eine Ergänzung für die Toolbox. Er nennt die erledigten Kapitel zu HTML und CSS und das nächste.

Eine README beschreibt den aktuellen Stand des Projekts. Ändert sich der Aufbau, gehört die Änderung der README in denselben Commit. Die folgenden Kapitel nennen diesen Schritt jeweils vor dem letzten Commit.

3.2 Markdown-Grundlagen

Markdown kennzeichnet die Struktur eines Textes mit Zeichen im Text selbst. Der Quelltext bleibt auch ohne Darstellung lesbar. Dieses Skript folgt der Spezifikation [CommonMark](#). Tabellen stammen aus der Erweiterung [GitHub Flavored Markdown](#) (GFM).

Element	Quelltext	Regel
Überschrift	# Titel, ## Abschnitt bis #####	Nach den Zeichen # steht ein Leerzeichen. Die README hat genau eine Überschrift mit #.
Absatz	Text, danach eine Leerzeile	Ein einfacher Zeilenumbruch beginnt keinen neuen Absatz.
Liste	- Eintrag	Jeder Eintrag beginnt in einer neuen Zeile.
Nummerierte Liste	1. Schritt	Die Nummerierung beginnt mit der Zahl des ersten Eintrags und steigt um eins je Eintrag. Die Zahlen der folgen-

Element	Quelltext	Regel
		den Einträge ändern sie nicht: 3. A und 1. B ergeben 3 und 4.
Link	[Linktext](https://...), [Linktext] (datei.md)	Relative Pfade wie in Dateisystem & Pfade (Kapitel 1.1).
Code im Text	<code>`index.html`</code>	für Dateinamen, Befehle und Code
Code-Block	eine Zeile mit mindestens drei Backticks ``` davor und danach	Zeilenumbrüche und Leerzeichen bleiben erhalten. Nach den öffnenden Backticks steht optional die Sprache, zum Beispiel bash für Befehle im Terminal.
Tabelle	A B , darunter --- ---	Die zweite Zeile trennt die Spaltenköpfe vom Inhalt. Die senkrechten Striche müssen nicht untereinander stehen.

Zwischen zwei Blöcken (Überschrift, Absatz, Liste, Code-Block, Tabelle) steht eine Leerzeile.

Markdown Preview aus [VS Code](#) (Kapitel 1.2) zeigt das Ergebnis neben dem Quelltext.

3.3 README anlegen

Ausgangsstand ist die Toolbox aus [Git & GitHub](#) (Kapitel 1.4) mit `.gitignore`, `learning-notizen.md` und drei Commits. Ist ein anderes Projekt offen, öffne die Toolbox mit **File → Open Folder...**

Lege im **Explorer** von VS Code mit **New File...** im Ordner toolbox neben `learning-notizen.md` die Datei `README.md` an. Kopiere den Inhalt der folgenden vier Code-Blöcke in dieser Reihenfolge in die Datei. Lass zwischen zwei Code-Blöcken eine Leerzeile. Beim Aufbau gehören die Zeilen mit drei Backticks zum Inhalt.

Titel und Zweck. Der Titel ist der Name des Projekts. Der Absatz darunter nennt den Zweck in einem Satz.

```
# Toolbox
```

```
Bauteile für das Web: moderne HTML- und CSS-Patterns zum Nachschlagen - gebaut mit genau diesen Patterns.
```

Aufbau. Der Strukturbaum zeigt jede Datei mit einer kurzen Beschreibung nach dem Pfeil ←. Er steht in einem Code-Block, weil Markdown die Zeilen außerhalb eines Code-Blocks zu einem Absatz zusammenfasst.

```
## Struktur
```

```
...
```

```
toolbox/
```

```
├── README.md           ← diese Datei
```

```
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
```

```
└── .gitignore          ← Systemdateien, die nicht ins Repository gehören
```

```
...
```

Fortschritt. Die Tabelle nennt das nächste Kapitel. Am Ende jedes Kapitels zu HTML und CSS setzt du dessen Zeile auf „erledigt“ und ergänzt eine Zeile für das nächste.

```
## Fortschritt

| Kapitel | Status |
| --- | --- |
| Das Skelett | offen |
```

Quellen. Die Liste nennt die Quellen, auf denen das Projekt aufbaut.

```
## Quellen

- [MDN Web Docs](https://developer.mozilla.org/) - Primärquelle für alle Einträge
```

Speichere die Datei. Der Tab README.md zeigt danach keinen Punkt mehr.

3.4 Tastenkürzel

Aktion	Windows	macOS
Speichern	Strg + S	Cmd + S
Backtick `	Shift + Taste rechts neben ß, danach Leertaste	Shift + Taste rechts neben ß, danach Leertaste
Senkrechter Strich	AltGr + <	Option + 7

4. Verstehen

Lege im Lernprotokoll die Überschrift **## Markdown & README (Datum)** an. Der Ablauf der Beobachtung ist: Aufgabe ausführen, Ergebnis und Erklärung notieren.

Beobachtung: Eine fremde README

MarkItDown ist ein Werkzeug von Microsoft. Es wandelt Dateien wie PDF, Word und PowerPoint in Markdown um.

- **Aufgabe:** Lies die README von MarkItDown unter der Dateiliste bis einschließlich des Unterabschnitts **Command-Line** im Abschnitt **Usage**. Die Befehle musst du weder verstehen noch ausführen.
- **Notieren:** Halte zu Zweck, Start und Aufbau aus Was eine README beantwortet fest, an welcher Stelle die README den Teil beantwortet: Abschnitt oder Absatz. Notiere, welcher Teil fehlt und wer ihn braucht: Personen, die das Werkzeug verwenden, oder Personen, die am Code mitarbeiten.

→ Auflösung 1 am Ende des Kapitels

Commit

Der Commit enthält die README und die Notizen zu diesem Kapitel. Der Typ docs kennzeichnet Änderungen an der Dokumentation (siehe Git & GitHub (Kapitel 1.4)).

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Changes) über der Liste Changes
Prüfen	<code>git status</code> zeigt README.md und learning-notizen.md unter Changes to be committed	Beide Dateien stehen unter Staged Changes
Commit	<code>git commit -m "docs: README anlegen"</code>	Nachricht eingeben → Commit
Übertragen	<code>git push</code>	Sync Changes

5. Prüfen

- ☐ README.md liegt im Ordner toolbox neben learning-notizen.md und enthält Titel, Zweck, Struktur, Fortschritt und Quellen.
- ☐ Nach jedem # einer Überschrift steht ein Leerzeichen. Zwischen zwei Blöcken steht eine Leerzeile.
- ☐ Der Strukturbaum steht zwischen zwei Zeilen mit drei Backticks und nennt README.md, learning-notizen.md und .gitignore.
- ☐ Unter der Kopfzeile der Tabelle im Abschnitt ## Fortschritt steht die Trennzeile | --- | --- |.
- ☐ `git log --oneline` zeigt als neuesten Commit docs: README anlegen. Die Seite des Repositories auf GitHub zeigt nach dem Neuladen **4 Commits**.
- ☐ learning-notizen.md enthält zur Beobachtung Ergebnis und Erklärung.

Zielstand der Toolbox:

```

toolbox/
├── .git/
├── .gitignore
├── learning-notizen.md
└── README.md

```

Quellen

- [CommonMark Spec 0.31.2](#)
- [CommonMark – Markdown Reference](#)
- [GitHub Flavored Markdown Spec](#)
- [GitHub Docs – About the repository README file](#)
- [GitHub Docs – Basic writing and formatting syntax](#)
- [VS Code – Markdown](#)
- [MarkItDown – Microsoft](#)

Auflösungen

Auflösung 1

Zu Beobachtung: Eine fremde README

Stand Oktober 2026:

Teil	Antwort in der README
Zweck	Absatz nach dem Hinweis IMPORTANT , ohne eigene Überschrift: ein Werkzeug in Python, das Dateien für die Textanalyse in Markdown umwandelt. Why Markdown? begründet das Format der Ausgabe.
Start	Abschnitte Prerequisites , Installation und Usage → Command-Line ; jeder Befehl steht in einem Code-Block, im Quelltext (Raw) mit der Sprache bash
Aufbau	kein eigener Abschnitt; Ordner wie <code>packages/markitdown</code> erscheinen nur in Befehlen und Verweisen

Die README richtet sich an Personen, die das Werkzeug verwenden. Für sie genügen Zweck und Start. Wer am Code mitarbeitet, muss den Aufbau aus den Ordnern des Repositorys erschließen. Die README der Toolbox beschreibt den Aufbau, weil Lehrende die Dateien des Projekts prüfen.

1.6 Git in der Gruppe

Voraussetzungen: Git & GitHub (Kapitel 1.4), Markdown & README (Kapitel 1.5)

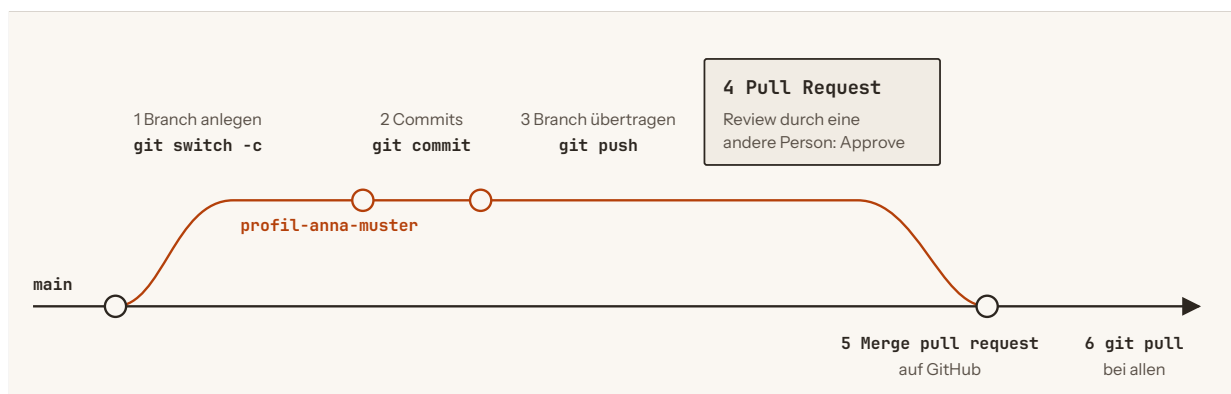
Lernziele

- Ein gemeinsames Repository einrichten und eine Änderung auf einem eigenen Branch über einen Pull Request nach `main` bringen, im Terminal und in VS Code.
- Einen Pull Request einer anderen Person prüfen und freigeben, den eigenen Pull Request nach der Freigabe mergen und `main` danach lokal aktualisieren.
- Einen Merge-Konflikt erkennen, in VS Code auflösen und den Pull Request danach mergen.

1. Vorschau

Zielstand nach diesem Kapitel: Die Gruppe arbeitet in einem gemeinsamen privaten Repository projekt auf GitHub. Jede Person hat darin mindestens drei Pull Requests gemergt, jeden nach der Freigabe durch eine andere Person. Die README hält die Regeln der Zusammenarbeit fest.

Das Modell hinter den Schritten dieses Kapitels, der GitHub Flow:



2. Warum

In einer Gruppe ändern mehrere Personen gleichzeitig dieselben Dateien. Ohne festen Ablauf überschreiben sie gegenseitig ihre Stände oder übernehmen fehlerhafte Änderungen ungeprüft. Der **GitHub Flow** legt jede Aufgabe auf einen eigenen Branch und übernimmt sie erst nach der Prüfung durch eine zweite Person nach `main`. Softwareteams arbeiten in Varianten dieses Ablaufs; ein Pull Request mit Review vor jedem Merge ist dort üblich.

Weiterführend: [GitHub Docs – GitHub flow](#) · [Pro Git, Kapitel 3: Git Branching](#)

3. Einrichten

Dieses Kapitel arbeitet in einem neuen Repository der Gruppe. Die Toolbox bleibt Einzelarbeit; sie bekommt am Ende nur die Notizen im Lernprotokoll. Wie in [Git & GitHub](#) (Kapitel 1.4) nennt das Kapitel jeden Git-Schritt als Befehl im **Terminal** und als Aktion in **VS Code**. Schritte auf GitHub führst du im Browser aus.

Die Beispiele verwenden den Benutzernamen `anna-muster`. Setze an diesen Stellen deinen eigenen Benutzernamen ein.

3.1 Die Regeln der Gruppe

Die Gruppe arbeitet nach fünf Regeln:

Regel	Grund
1. Niemand committet direkt auf <code>main</code> .	<code>main</code> enthält nur geprüfte Stände. Jede Person kann jederzeit von <code>main</code> aus weiterarbeiten.
2. Jede Aufgabe bekommt einen eigenen Branch vom aktuellen <code>main</code> .	Aufgaben laufen parallel, ohne die Stände der anderen zu verändern.
3. Jeder Branch kommt über einen Pull Request nach <code>main</code> .	Der Pull Request zeigt alle Änderungen einer Aufgabe an einer Stelle.
4. Eine andere Person prüft den Pull Request und gibt ihn frei. Wer den Pull Request geöffnet hat, mergt ihn.	Eine zweite Person sieht jede Änderung. Den Zeitpunkt des Merge bestimmt die Person, die die Änderung kennt.
5. Wer den Pull Request gemergt hat, löscht den Branch. Alle aktualisieren <code>main</code> mit <code>git pull</code> .	Jeder neue Branch beginnt auf dem neuesten Stand.

GitHub setzt diese Regeln in der Einrichtung dieses Kapitels nicht durch. Jede Person der Gruppe kann direkt auf `main` pushen und ohne Freigabe mergen. Die Regeln sind eine Vereinbarung der Gruppe; [3.10](#) schreibt sie in die README.

3.2 Repository anlegen und einladen

Eine Person der Gruppe legt das Repository an. Sie ist **Owner** des Repositorys.

1. Wähle auf [github.com](#) **+** → **New repository**.
2. Setze den Namen auf `projekt` und die Sichtbarkeit auf **Private**.
3. Aktiviere die Option für eine README. Das Repository enthält dann einen ersten Commit mit der Datei `README.md`.
4. Wähle **Create repository**.
5. Wähle im Repository **Settings** → **Collaborators** (Bereich **Access**) → **Add people**.
6. Suche eine Person der Gruppe über ihren Benutzernamen und wähle **Add ... to projekt**. Wiederhole den Schritt für jede Person der Gruppe.

Die eingeladenen Personen erhalten eine E-Mail und nehmen die Einladung über den Link darin an. Danach können alle Personen der Gruppe in das Repository pushen.

3.3 Repository klonen

Jede Person legt eine Kopie des Repositorys auf ihrem Rechner an. Diesen Schritt nennt Git Clone. Der Klon enthält alle Commits und ist mit dem Repository auf GitHub als Remote origin verbunden.

Kopiere die Adresse auf der Seite des Repositorys über **Code** → **HTTPS**. Sie enthält den Benutzernamen des Owners.

Achtung: Lege den Klon neben dem Ordner toolbox ab, nicht darin. Liegt projekt in der Toolbox, meldet git status in der Toolbox den Ordner projekt/ unter Untracked files.

Terminal	VS Code
Wechsle mit <code>cd ..</code> aus der Toolbox in den Ordner darüber. Führe <code>git clone https://github.com/anna-muster/projekt.git</code> aus.	Command Palette → Git: Clone → Adresse einfügen → Enter → Ordner über toolbox wählen → Select as Repository Destination

Git antwortet mit `Cloning into 'projekt' ...` und legt den Ordner projekt an. Der Klon verwendet die Anmeldung bei GitHub aus Git & GitHub (Kapitel 1.4).

Öffne den Ordner projekt als Workspace. Im Terminal-Weg wählst du **File** → **Open Folder...**. VS Code fragt nach dem Klonen: „Would you like to open the repository?“ Wähle **Open**. Alle weiteren Schritte laufen in diesem Workspace.

3.4 Branch anlegen

Der Name eines Branch beschreibt die Aufgabe in kebab-case. Der erste Branch heißt profil- und dein Benutzername, zum Beispiel profil-anna-muster.

Terminal	VS Code
<code>git switch -c profil-anna-muster</code>	Branch-Name main in der Status Bar → Create new branch... → Name eingeben → Enter

Git antwortet mit `Switched to a new branch 'profil-anna-muster'`. Die **Status Bar** von VS Code zeigt den neuen Branch. Der Branch beginnt beim aktuellen Commit von main. Commits entstehen ab jetzt auf dem Branch; main bleibt unverändert.

3.5 Commit und Push des Branch

Lege im Workspace den Ordner team an und darin eine Datei mit deinem Benutzernamen, zum Beispiel anna-muster.md. Übernimm diesen Inhalt und setze deine Angaben ein:

```
# Anna Muster

- GitHub: anna-muster
- Aufgabe im Projekt: offen
- Erfahrung mit HTML und CSS: keine
- Erfahrung mit Git: Kapitel Git & GitHub
```

Jede Person der Gruppe ändert eine eigene Datei. Die vier Pull Requests dieses Abschnitts lassen sich deshalb in beliebiger Reihenfolge mergen.

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Changes) über der Liste Changes
Prüfen	<code>git status</code> zeigt new file: team/anna-muster.md unter Changes to be committed	Die Datei steht unter Staged Changes
Commit	<code>git commit -m "docs: Profil von Anna Muster ergänzen"</code>	Nachricht eingeben → Commit
Übertragen	<code>git push -u origin profil-anna-muster</code>	Publish Branch

Der erste Push eines Branch legt ihn auf GitHub an. Git meldet * [new branch] profil-anna-muster → profil-anna-muster. Die Option -u verbindet den lokalen Branch mit dem Branch auf GitHub. Danach genügt `git push` bzw. **Sync Changes**.

3.6 Pull Request öffnen

Ein Pull Request ist eine Anfrage auf GitHub, die Commits eines Branch nach main zu übernehmen. Er zeigt alle Änderungen des Branch und sammelt Kommentare und Freigaben.

1. Öffne das Repository auf GitHub. Wähle im Hinweis über der Dateiliste **Compare & pull request**.
2. Prüfe die Zeile über dem Titel: **base** ist main, **compare** ist dein Branch.
3. Setze den Titel nach Conventional Commits (Kapitel 1.4), zum Beispiel docs: Profil von Anna Muster ergänzen.
4. Beschreibe im Feld darunter in einem Satz, was der Pull Request ändert.
5. Wähle rechts unter **Reviewers** eine Person der Gruppe.
6. Wähle **Create pull request**.

Fehlt der Hinweis, wähle **Pull requests** → **New pull request** und unter **compare** deinen Branch. GitHub nummeriert Pull Requests fortlaufend und listet sie im Tab **Pull requests**.

3.7 Review: prüfen und freigeben

Ein Review ist die Prüfung eines Pull Request durch eine andere Person. GitHub lässt die Freigabe des eigenen Pull Request nicht zu.

1. Öffne auf GitHub den Tab **Pull requests** und den Pull Request, für den du als Reviewer eingetragen bist.
2. Wähle den Tab **Files changed**. GitHub zeigt jede hinzugefügte und jede entfernte Zeile.
3. Prüfe den Pull Request gegen die Liste unten.
4. Wähle **Review changes** → **Approve** → **Submit review**.

Prüfe im Review:

- Der Pull Request enthält nur die Änderung, die sein Titel nennt.
- Die Commit-Nachrichten folgen Conventional Commits.
- Der Pull Request enthält keine Dateien des Betriebssystems wie .DS_Store (siehe Git & GitHub (Kapitel 1.4)).

Erfüllt der Pull Request einen Punkt nicht, wähle statt **Approve** die Option **Request changes** und beschreibe die nötige Änderung im Kommentar. Die Person committet die Korrektur auf

demselben Branch und überträgt sie mit `git push`. Der Pull Request zeigt den neuen Commit ohne weiteren Schritt.

3.8 Mergen

Öffne deinen Pull Request auf GitHub. Nach der Freigabe zeigt der Tab **Conversation** das Review mit dem Namen der prüfenden Person.

Achtung: Belasse die Methode im Menü neben der Schaltfläche **Merge pull request** auf **Create a merge commit**. Die anderen Methoden erzeugen keinen Merge-Commit.

1. Wähle unten im Tab **Conversation** die Schaltfläche **Merge pull request**.
2. Wähle **Confirm merge**.
3. Wähle **Delete branch**.

Die Schaltfläche **Merge pull request** erzeugt auf `main` einen Merge-Commit mit der Nachricht `Merge pull request #<Nummer> from <Owner>/<Branch>`. Die Nachricht erzeugt GitHub; sie folgt nicht Conventional Commits.

Delete branch löscht den Branch auf GitHub. Seine Commits bleiben über den Merge-Commit in `main` erhalten.

3.9 main aktualisieren

Den Merge-Commit enthält zunächst nur `main` auf GitHub. Nach jedem Merge, auch dem einer anderen Person, aktualisiert jede Person ihren lokalen `main`:

Schritt	Terminal	VS Code
Zu <code>main</code> wechseln	<code>git switch main</code>	Branch-Name in der Status Bar → <code>main</code>
Aktualisieren	<code>git pull</code>	Command Palette → Git: Pull
Eigenen Branch löschen	<code>git branch -d profil-anna-muster</code>	Command Palette → Git: Delete Branch... → Branch wählen

Pull holt die neuen Commits von GitHub und übernimmt sie in den aktuellen Branch. Ausgabe von `git pull` nach dem ersten Merge:

```
Updating 82f5be1..7f6dc8d
Fast-forward
 team/anna-muster.md | 6 ++++++
 1 file changed, 6 insertions(+)
 create mode 100644 team/anna-muster.md
```

Fast-forward bedeutet: Der lokale `main` hatte keine eigenen Commits, Git setzt ihn auf den Stand von GitHub. Regel 1 sorgt dafür, dass `git pull` auf `main` immer so endet. `git branch -d` antwortet mit `Deleted branch profil-anna-muster (was ...)`.

Der nächste Branch beginnt mit 3.4 auf dem aktualisierten `main`.

3.10 Regeln in die README

Eine Person der Gruppe hält die Regeln aus 3.1 in der README fest. Sie folgt dem Ablauf von Branch anlegen bis main aktualisieren mit dem Branch `readme-zusammenarbeit` und der Com-

mit-Nachricht docs: Regeln der Zusammenarbeit ergänzen. Ersetze den Inhalt von README.md durch:

```
# projekt

Repository der Gruppe. Wer zur Gruppe gehört, steht im Ordner `team`.

## Zusammenarbeit

1. Niemand committet direkt auf `main`.
2. Jede Aufgabe bekommt einen eigenen Branch vom aktuellen `main`.
3. Jeder Branch kommt über einen Pull Request nach `main`.
4. Eine andere Person prüft den Pull Request und gibt ihn frei. Wer den Pull Request geöffnet hat, mergt ihn.
5. Wer den Pull Request gemergt hat, löscht den Branch. Alle aktualisieren `main` mit `git pull`.

Commit-Nachrichten und Titel von Pull Requests folgen [Conventional Commits] (https://www.conventionalcommits.org/en/v1.0.0/).
```

3.11 Merge-Konflikt auflösen

Ändern zwei Branches dieselbe Stelle einer Datei, kann Git die Änderungen nicht automatisch zusammenführen. Das ist ein Merge-Konflikt. Der Pull Request meldet dann `This branch has conflicts that must be resolved` und bietet **Merge pull request** nicht an.

Den Konflikt löst die Person, die den Pull Request geöffnet hat, auf ihrem eigenen Branch:

Schritt	Terminal	VS Code
Zum eigenen Branch wechseln	<code>git switch <branch></code>	Branch-Name in der Status Bar → Branch wählen
Stand von GitHub abrufen	<code>git fetch</code>	Command Palette → Git: Fetch
main von GitHub übernehmen	<code>git merge origin/main</code>	Command Palette → Git: Merge... → origin/main

Fetch ruft die neuen Commits von GitHub ab und ändert keinen lokalen Branch. `git fetch` aktualisiert origin/main. `git merge origin/main` führt diesen Stand in deinen Branch. Bei einem Konflikt meldet Git:

```
Auto-merging team/anna-muster.md
CONFLICT (content): Merge conflict in team/anna-muster.md
Automatic merge failed; fix conflicts and then commit the result.
```

Git schreibt beide Fassungen der betroffenen Zeilen in die Datei, getrennt durch Konfliktmarker:

```
<<<<<<< HEAD
Zeilen aus deinem Branch
=====
Zeilen aus origin/main
>>>>>>> origin/main
```

So löst du den Konflikt auf:

1. Öffne die Datei. VS Code zeigt über jedem Konflikt die Aktionen **Accept Current Change** (dein Branch), **Accept Incoming Change** (origin/main) und **Accept Both Changes**.
2. Wähle eine Aktion oder bearbeite die Zeilen von Hand. Die Datei muss danach den gewünschten Inhalt haben und darf keinen Konfliktmarker mehr enthalten.
3. Speichere die Datei und übertrage die Auflösung:

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Merge Changes) über der Liste Merge Changes
Prüfen	<code>git status</code> zeigt <code>All conflicts fixed but you are still merging.</code>	Die Datei steht unter Staged Changes
Commit	<code>git commit -m "fix: Konflikt mit main auflösen"</code>	Nachricht eingeben → Commit
Übertragen	<code>git push</code>	Sync Changes

Der Pull Request meldet danach keinen Konflikt mehr. Das Review und der Merge folgen [3.7](#) und [3.8](#).

4. Verstehen

Öffne `learning-notizen.md` der Toolbox mit **File → Open File...** und lege die Überschrift `## Git in der Gruppe (Datum)` an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren. Die Experimente laufen im Repository projekt und ändern nur deine Datei im Ordner `team`. Ihre Änderungen bleiben in `main`.

Predict 1: Auf dem neuesten Stand

Ausgangsstand: Auf GitHub ist ein Pull Request gemergt, von dir oder einer anderen Person. Lokal hast du danach weder `git pull` noch `git fetch` ausgeführt.

- **Vorhersage:** Was meldet `git status` auf `main` in der zweiten Zeile?
- **Prüfen:**
 1. Wechsle zu `main` und führe `git status` aus.
 2. Führe `git fetch` aus und danach erneut `git status`.

→ [Auflösung 1](#) am Ende des Kapitels

Predict 2: Zwei Änderungen in einer Datei

Lege zwei Branches von `main` an, jeweils mit einem Commit in deiner Datei im Ordner `team`:

Branch	Änderung	Commit-Nachricht
<code>aufgabe-anna-muster</code>	Zeile - Aufgabe im Projekt: ... → - Aufgabe im Projekt: Startseite	<code>docs: Aufgabe von Anna Muster eintragen</code>
<code>erfahrung-anna-muster</code>	Zeile - Erfahrung mit HTML und CSS: ... → - Erfahrung mit HTML und CSS: ein Semester	<code>docs: Erfahrung von Anna Muster aktualisieren</code>

1. Lege den ersten Branch nach [3.4](#) an. Ändere die Zeile, erstelle den Commit und übertrage den Branch nach [3.5](#).

2. Wechsle zurück zu main. Lege den zweiten Branch an und verfähre ebenso.
 3. Öffne für beide Branches einen Pull Request mit einer Person der Gruppe als Reviewer.
 - **Vorhersage:** Die beiden Branches ändern verschiedene Zeilen derselben Datei. Lässt sich der zweite Pull Request mergen, nachdem der erste gemergt ist?
 - **Prüfen:** Merge den Pull Request zu aufgabe-anna-muster nach der Freigabe. Öffne danach den Pull Request zu erfahrung-anna-muster und betrachte den Bereich über der Schaltfläche **Merge pull request**.
- [Auflösung 2](#) am Ende des Kapitels

Break-it 1: Konfliktmarker im Commit

Führe für den Branch erfahrung-anna-muster die drei Schritte der ersten Tabelle aus [3.11](#) im Terminal aus. Ändere die Datei danach nicht.

- **Vorhersage:** Du überträgst die Datei mit den Konfliktmarkern unverändert in einen Commit. Lehnt Git den Commit ab, und was zeigt der Pull Request danach?
- **Prüfen:**

Achtung: Merge den Pull Request in diesem Zustand nicht.

 1. Führe im Terminal `git add ., git commit -m "fix: Konflikt mit main auflösen"` und `git push` aus.
 2. Öffne den Pull Request auf GitHub. Betrachte den Bereich über **Merge pull request** und den Tab **Files changed**.

→ [Auflösung 3](#) am Ende des Kapitels

Commit

Öffne die Toolbox mit **File → Open Folder...** Schließe die Notizen zu diesem Kapitel ab und übertrage sie. Formuliere die Nachricht selbst nach Conventional Commits.

Schritt	Terminal	VS Code
In die Staging Area	<code>git add .</code>	+ (Stage All Changes) über der Liste Changes
Prüfen	<code>git status</code> zeigt die Dateien unter Changes to be committed	Die Dateien stehen unter Staged Changes
Commit	<code>git commit -m "<type>: <description>"</code>	Nachricht eingeben → Commit
Übertragen	<code>git push</code>	Sync Changes

5. Prüfen

- ☐ Der Tab **Pull requests** → **Closed** zeigt für jede Person der Gruppe mindestens drei gemergte Pull Requests, jeden mit der Freigabe einer anderen Person.
- ☐ Die README auf GitHub zeigt den Abschnitt „Zusammenarbeit“ mit fünf Regeln.
- ☐ `git log --oneline --graph` zeigt in projekt Merge-Commits mit der Nachricht `Merge pull request #...`
- ☐ `git branch` zeigt in projekt nur main.

- ☐ Nach `git fetch` meldet `git status` in projekt auf main die Zeilen `Your branch is up to date with 'origin/main'`. und `nothing to commit, working tree clean`.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.

Zielstand des Repositorys projekt für eine Gruppe mit vier Personen:

```
projekt/  
├── .git/  
├── README.md  
└── team/  
    ├── anna-muster.md  
    ├── ben-beispiel.md  
    ├── carla-probe.md  
    └── david-test.md
```

Quellen

- [GitHub Docs – GitHub flow](#)
- [GitHub Docs – Inviting collaborators to a personal repository](#)
- [GitHub Docs – Creating a pull request](#)
- [GitHub Docs – Reviewing proposed changes in a pull request](#)
- [GitHub Docs – Merging a pull request](#)
- [VS Code – Merge conflicts](#)
- [VS Code – Branches and worktrees](#)
- [Pro Git \(Deutsch\), Kapitel 3: Git Branching](#) – Scott Chacon, Ben Straub

Auflösungen

Auflösung 1

Zu Predict 1: Auf dem neuesten Stand

Vor dem Abruf meldet `git status` die Zeile `Your branch is up to date with 'origin/main'`.

Nach `git fetch` lautet die Zeile `Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded`. Ein Pull Request mit einem Commit ergibt zwei neue Commits auf `main`: den Commit des Branch und den Merge-Commit.

`git status` fragt GitHub nicht ab. Es vergleicht `main` mit `origin/main`, dem Stand von GitHub beim letzten `git fetch` oder `git pull`. „Up to date“ bedeutet deshalb: gleich dem zuletzt abgerufenen Stand. Regel 5 verlangt deshalb `git pull`, bevor ein neuer Branch beginnt.

Führe `git pull` aus.

Auflösung 2

Zu Predict 2: Zwei Änderungen in einer Datei

Der zweite Pull Request meldet `This branch has conflicts that must be resolved`. Die beiden geänderten Zeilen liegen direkt untereinander. Git erkennt die Position einer Änderung an den unveränderten Zeilen davor und danach. Zwischen den beiden Änderungen liegt keine unveränderte Zeile, deshalb behandelt Git sie als eine Stelle. Liegt mindestens eine unveränderte Zeile dazwischen, führt Git beide Änderungen automatisch zusammen.

Lass den Pull Request offen. Break-it 1 setzt ihn fort.

Auflösung 3

Zu Break-it 1: Konfliktmarker im Commit

Git nimmt den Commit an. `git add` kennzeichnet die Datei als aufgelöst, ihren Inhalt prüft Git nicht. Die Konfliktmarker sind für Git gewöhnliche Textzeilen. Der Pull Request meldet keinen Konflikt mehr und bietet **Merge pull request** an. **Files changed** zeigt die Marker als neue Zeilen; ein Merge würde sie nach `main` übertragen. Beim Stagen einer Datei mit Konfliktmarkern zeigt VS Code eine Rückfrage mit dem Text „Are you sure you want to stage `anna-muster.md` with merge conflicts?“.

Korrigiere den Branch. Entferne die Marker und stelle beide Änderungen her: - Aufgabe im Projekt: Startseite und - Erfahrung mit HTML und CSS: ein Semester. Keine der drei Aktionen aus Merge-Konflikt auflösen ergibt diesen Inhalt. **Accept Both Changes** übernimmt beide Fassungen und verdoppelt die Zeilen. Übertrage die Korrektur mit der Commit-Nachricht `fix: Konfliktmarker entfernen` und `git push`. **Files changed** zeigt danach nur die geänderte Zeile zur Erfahrung. Merge den Pull Request nach der Freigabe und aktualisiere `main` nach 3.9. Lösche dabei beide Branches dieses Experiments.



Anhang



A Glossar

Begriffe aus dem Skript, alphabetisch sortiert. Englische Fachbegriffe bleiben englisch; die deutsche Übersetzung steht in der Definition. Die Kapitel verlinken auf diese Einträge.

Arbeitsweise

Lernprotokoll (`learning-notizen.md`)

Definition: Datei im Projekt der Toolbox. Sie enthält je Experiment die Vorhersage, das Ergebnis und die Erklärung. Sie ist der Nachweis des Verstehens und gehört nicht auf die Website.

Siehe auch: [Toolbox](#)

Lernziele (Kapitel)

Definition: Drei Stichpunkte am Anfang jedes Kapitels. Sie nennen, was du nach dem Kapitel kannst. Anhand der Lernziele entscheidest du, welche Teile du bearbeitest.

Pfad-Training

Definition: Übungsprojekt aus dem Material des Kapitels *Dateisystem & Pfade* mit drei Seiten und absichtlichen Pfadfehlern. Es verbindet die Kapitel *Dateisystem & Pfade*, *VS Code* und *Browser & DevTools*.

Siehe auch: [Pfad](#)

Toolbox

Definition: Das Lernprojekt der Kapitel zu HTML und CSS: eine Website, die Web-Patterns dokumentiert und mit genau diesen Patterns gebaut ist. Unterzeile „Bauteile für das Web“. Die Referenz-Toolbox zeigt die vollständige Toolbox.

Siehe auch: [Lernprotokoll](#)

Tooling

Definition: Das Handwerkszeug der Softwareentwicklung: Dateisystem, Editor, Browser-Dev-Tools und Git. Die Kapitel zu Tooling stehen vor den Kapiteln zu HTML und CSS.

Fachbegriffe

@starting-style

Definition: At-Rule mit Startwerten für eine Transition. Sie gilt, wenn ein Element zum ersten Mal dargestellt wird, zum Beispiel ein Popover beim Öffnen nach `display: none`. Ohne sie fehlt der Wert vor der Änderung, und die Transition findet nicht statt.

Siehe auch: [Transition](#), [Popover](#), [MDN Web Docs](#)

@supports

Definition: At-Rule mit einer Bedingung. Die Regeln im Block gelten nur, wenn der Browser die Deklaration in der Bedingung kennt; mit `not` nur, wenn er sie nicht kennt. Eine Bedingung mit einer Custom Property trifft immer zu, weil eine Custom Property jeden Wert annimmt.

Siehe auch: [Progressive Enhancement](#), [Ersatzwert](#), [MDN Web Docs](#)

:has()

Definition: Pseudoklasse, die ein Element trifft, wenn ein Nachfahre dem Selektor in den Klammern entspricht. Beginnt der Selektor mit `+`, gilt er für das folgende Geschwister. `html:has(body > nav:popover-open)` trifft `<html>`, solange die Sidebar als Popover offen ist.

Siehe auch: [Selektor](#), [Popover](#), [MDN Web Docs](#)

Accessibility Tree

Definition: Vom Browser aus dem DOM abgeleitete Struktur, die assistiven Technologien (z. B. Screenreadern) Rolle, Name und Zustand jedes Elements bereitstellt. In den DevTools einsehbar.

Siehe auch: [DOM](#), [Landmark](#), [MDN Web Docs](#)

Akkordeon (exklusiv)

Definition: Mehrere Elemente `<details>` mit demselben Wert im Attribut `name`. Der Browser hält höchstens einen Eintrag der Gruppe offen, auch beim Laden der Seite. Die Zeile `<summary>` ist ein Tab-Stopp mit dem Zustand `expanded`.

Siehe auch: [Accessibility Tree](#), [MDN Web Docs](#)

Alt-Text

Definition: Der Text im Attribut `alt` eines `<img>`. Er ersetzt das Bild: Screenreader lesen ihn vor, und der Browser zeigt ihn an, wenn die Datei nicht lädt. Er ist der Name des Bildes im Accessibility Tree. Ein dekoratives Bild erhält `alt=""` und fehlt dann im Accessibility Tree.

Siehe auch: [Accessibility Tree](#), [Caption](#), [W3C Web Accessibility Initiative](#)

aria-current

Definition: Attribut, das in einer Gruppe verwandter Elemente das aktuelle kennzeichnet. Der Wert `page` kennzeichnet in einer Navigation den Link, der auf die geöffnete Seite verweist. Das Stylesheet der Toolbox leitet die sichtbare Markierung aus dem Attribut ab.

Siehe auch: [Main Navigation](#), [Breadcrumbs](#), [MDN Web Docs](#)

Anchor

Definition: Stelle einer Seite, auf die ein Link verweist: ein Element mit dem Attribut `id`. Der Link nennt `#` und die `id` (`href="#struktur"`), auf einer anderen Seite mit Pfad davor (`../index.html#struktur`).

Siehe auch: [Linktext](#), [Skip-Link](#), [MDN Web Docs](#)

Anfangswert

Definition: Wert, den CSS für eine Eigenschaft festlegt. Er gilt, wenn keine Deklaration die Eigenschaft für das Element festlegt und die Eigenschaft nicht vererbt wird, zum Beispiel `transparent` für `background-color` oder `none` für `outline`.

Siehe auch: [Vererbung](#), [Custom Property](#), [MDN Web Docs](#)

Attribut

Definition: Angabe im öffnenden Tag eines Elements, geschrieben als Name und Wert: `lang="de"`, `href="index.html"`. Ein Attribut ergänzt oder steuert das Element.

Siehe auch: [Element](#), [Tag](#), [MDN Web Docs](#)

Audit

Definition: Prüfung einer Seite gegen eine Liste von Erfolgskriterien. Das Audit-Protokoll (`audit.md`) nennt je Kriterium Stufe, Prüfung, Ergebnis mit Stelle und Maßnahme.

Siehe auch: [Erfolgskriterium](#), [WCAG](#)

Baseline

Definition: Einstufung eines Features der Web-Plattform nach seiner Verfügbarkeit in den verbreiteten Browsern Chrome, Edge, Firefox und Safari. `Limited availability`: Mindestens ein verbreiteter Browser unterstützt es nicht. `Newly available`: Die aktuelle Version jedes verbreiteten Browsers unterstützt es. `Widely available`: seit 30 Monaten `Newly available`. Der Name des Programms und die Namen der Stufen werden nicht übersetzt.

Siehe auch: [Progressive Enhancement](#), [Web Platform Status](#), [Can I use](#)

Box Model

Definition: Modell, nach dem der Browser jedes Element als Box darstellt: Inhalt, padding (Innenabstand), border (Rahmen) und margin (Außenabstand). `box-sizing` legt fest, ob `width` nur den Inhalt (`content-box`) oder die Box bis einschließlich border (`border-box`) meint.

Siehe auch: [Margin Collapsing](#), [MDN Web Docs](#)

Branch

Definition: Benannte Entwicklungslinie in einem Git-Repository. Ein neues Repository hat einen Branch, in diesem Skript `main`; Commits entstehen auf dem aktuellen Branch.

Siehe auch: [Commit](#), [Repository](#)

Breadcrumbs

Definition: Navigation, die die Lage einer Seite innerhalb der Website nennt: eine geordnete Liste (`<ol>`) von Links von der Startseite bis zur aktuellen Seite. Der englische Begriff wird nicht übersetzt. Der letzte Link trägt `aria-current="page"`.

Siehe auch: [aria-current](#), [Pattern](#), [W3C Web Accessibility Initiative](#)

Breakpoint

Definition: Breite des Viewports, ab der die Bedingung einer Media Query zutrifft. Die Toolbox hat einen Breakpoint bei 64rem: Ab dieser Breite steht die Sidebar als Spalte neben dem Hauptinhalt.

Siehe auch: [Media Query](#), [Mobile-first](#)

Caption

Definition: Sichtbare Beschriftung eines Bildes im Element `<figcaption>` innerhalb von `<figure>` oder einer Tabelle im Element `<caption>`. Die Caption eines Bildes ergänzt es für alle; der Alt-Text ersetzt es. Die Caption einer Tabelle ist ihr Name im Accessibility Tree.

Siehe auch: [Alt-Text](#), [Header-Zelle](#), [MDN Web Docs](#)

Cascade Layers

Definition: Ebenen eines Stylesheets, festgelegt mit `@layer`. Eine Regel in einer späteren Ebene gewinnt gegen eine Regel in einer früheren Ebene, unabhängig von der Spezifität; eine Regel außerhalb jeder Ebene gewinnt gegen alle Ebenen. Im Fließtext heißt die einzelne Schicht „Ebene“. Die Toolbox hat die Ebenen `tokens`, `base`, `layout`, `components`, `overrides`.

Siehe auch: [Kaskade](#), [Spezifität](#), [MDN Web Docs](#)

clamp()

Definition: CSS-Funktion mit drei Angaben: Minimum, Wert und Maximum. Der Wert wächst zum Beispiel mit der Breite des Viewports (`vw`) und bleibt zwischen den zwei Grenzen. Eine Größe ändert sich so ohne Breakpoint.

Siehe auch: [Breakpoint](#), [Custom Property](#)

Clone

Definition: Kopie eines Repositorys von einem Remote auf den eigenen Rechner mit `git clone`, in VS Code **Git: Clone**. Der Klon enthält alle Commits und ist mit dem Remote als `origin` verbunden.

Siehe auch: [Remote](#), [Repository](#)

Code-Block

Definition: Block in Markdown zwischen zwei Zeilen mit mindestens drei Backticks; Zeilenumbrüche und Leerzeichen bleiben erhalten. Nach den öffnenden Backticks steht optional die Sprache. In HTML entspricht er `<pre><code>`.

Siehe auch: [Markdown](#)

Color Scheme

Definition: Helles oder dunkles Farbschema einer Seite: Light Mode oder Dark Mode. Die Eigenschaft `color-scheme` nennt die Color Schemes, die ein Element unterstützt; die Media Query `prefers-color-scheme` fragt ab, welches die Person im Betriebssystem eingestellt hat. Der englische Begriff wird nicht übersetzt.

Siehe auch: [light-dark\(\)](#), [Media Query](#), [MDN Web Docs](#)

Commit

Definition: Gespeicherter Stand eines Git-Repositorys mit Nachricht, Autor:in und eindeutiger Kennung (Hash). Entsteht in zwei Schritten: `git add` (in die Staging Area), `git commit`.

Siehe auch: [Repository](#), [Staging Area](#)

Container Query

Definition: Bedingung im Stylesheet (`@container`), unter der Regeln gelten, zum Beispiel ab einer Breite des Query Containers. Eine Komponente passt sich damit an den Platz an ihrem Ort an, nicht an den Viewport.

Siehe auch: [Query Container](#), [Media Query](#)

Container und Item

Definition: Ein Container ist ein Element mit `display: flex` (Flex-Container) oder `display: grid` (Grid-Container). Seine Kinder sind die Items. Eigenschaften des Containers legen die Anordnung aller Items fest; Enkel sind keine Items.

Siehe auch: [Flexbox](#), [Grid](#)

Conventional Commits

Definition: Konvention für Commit-Nachrichten der Form `<type>: <description>`, z. B. `fix: Pfad zum Logo korrigieren`. Die Spezifikation definiert `feat` und `fix` und lässt weitere Typen zu.

Siehe auch: [Commit](#), [Conventional Commits 1.0.0](#)

CSS

Definition: Cascading Style Sheets – regelbasierte Sprache für die Darstellung von HTML. Ein Stylesheet besteht aus Regeln; der Browser wendet jede Regel auf alle Elemente an, auf die ihr Selektor zutrifft.

Siehe auch: [Regel](#), [Stylesheet](#), [Kaskade](#), [MDN Web Docs](#)

Custom Property

Definition: Eigenschaft mit frei gewähltem Namen, der mit `--` beginnt (`--accent-hue: 45;`). Die Funktion `var()` setzt ihren Wert in eine Deklaration ein. Custom Properties werden vererbt; der Browser prüft den Wert erst beim Einsetzen.

Siehe auch: [Deklaration](#), [MDN Web Docs](#)

Dateiendung

Definition: Teil des Dateinamens nach dem letzten Punkt (.html, .css, .md). Bestimmt, wie Programme die Datei behandeln. Windows und macOS blenden sie standardmäßig aus – für Webprojekte immer einblenden.

Deklaration

Definition: Paar aus Eigenschaft und Wert in einer Regel eines Stylesheets, z. B. `font-size: var(--text-xs);`. Eine Regel besteht aus einem Selektor und einem Block von Deklarationen.

Siehe auch: [Selektor](#), [Kaskade](#)

DevTools

Definition: In den Browser eingebaute Entwicklerwerkzeuge (Elements, Console, Network, Accessibility ...), die zeigen, was der Browser geladen und aus dem Code gemacht hat. Änderungen dort wirken nur im DOM, nicht in der Datei.

Siehe auch: [DOM](#)

DOM

Definition: Document Object Model – die Baumstruktur, die der Browser aus dem HTML-Quelltext erzeugt. Kann vom Quelltext abweichen, weil der Browser Fehler stillschweigend korrigiert.

Siehe auch: [Accessibility Tree](#)

Element

Definition: Baustein eines HTML-Dokuments aus öffnendem Tag, Inhalt und schließendem Tag, zum Beispiel `<title>Toolbox</title>`. Leere Elemente wie `<meta>` und `<link>` haben keinen Inhalt und keinen schließenden Tag.

Siehe auch: [Tag](#), [Attribut](#), [DOM](#), [MDN Web Docs](#)

Erfolgskriterium

Definition: Prüfbare Aussage der WCAG mit Nummer und Stufe (A, AA, AAA), zum Beispiel 1.4.3 Contrast (Minimum), Stufe AA. Die Nummer nennt Prinzip, Richtlinie und Kriterium. Englisch: Success Criterion.

Siehe auch: [WCAG](#), [Audit](#), [W3C Web Accessibility Initiative](#)

Ersatzwert

Definition: Wert für den Fall, dass ein anderer Wert nicht gilt. Als zweite Angabe in `var()` gilt er, wenn die Custom Property nicht definiert ist. Als Deklaration vor einer Deklaration mit einem neueren Feature gilt er in Browsern ohne das Feature. Vor einer Deklaration mit `var()` wirkt er nicht: Der Browser prüft den eingesetzten Wert erst nach der Kaskade.

Siehe auch: [Custom Property](#), [@supports](#), [Kaskade](#)

Fetch

Definition: Abruf neuer Commits von einem Remote mit `git fetch`. Fetch aktualisiert `origin/main` und ändert keinen lokalen Branch.

Siehe auch: [Pull](#), [Remote](#)

Flexbox

Definition: Layout-Verfahren für Items in einer Richtung (Hauptachse): in einer Zeile oder in einer Spalte, mit Umbruch (`flex-wrap`) und Verteilung des freien Platzes je Zeile (`justify-content`). Der Inhalt der Items bestimmt die Aufteilung. Die Querachse verläuft senkrecht zur Hauptachse.

Siehe auch: [Container und Item](#), [Grid](#), [Gap](#), [MDN Web Docs](#)

Fokus

Definition: Das Element, an das Eingaben über die Tastatur gehen. Ein Klick oder die Taste Tab setzt ihn; der Browser zeigt ihn meist als Rahmen um das Element.

Siehe auch: [Tab-Stopp](#)

Gap

Definition: Abstand zwischen den Items eines Flex- oder Grid-Containers, festgelegt mit `gap`. Das Gap steht nur zwischen Items, nicht am Rand des Containers.

Siehe auch: [Flexbox](#), [Grid](#)

Git

Definition: Werkzeug zur Versionskontrolle: speichert Stände (Commits) eines Projekts lokal im Ordner `.git`. GitHub, GitLab oder Codeberg sind Plattformen, auf denen Repositories online liegen.

Siehe auch: [Commit](#), [Remote](#)

Grid

Definition: Layout-Verfahren mit Spalten und Zeilen, die der Container für alle Items festlegt (`grid-template-columns`, `grid-template-rows`). Die Einheit `fr` ist ein Anteil am freien Platz; `minmax()` nennt die kleinste und die größte Größe einer Spalte oder Zeile.

Siehe auch: [Grid Area](#), [Flexbox](#), [Container und Item](#), [MDN Web Docs](#)

Grid Area

Definition: Bereich eines Grids mit einem Namen. `grid-template-areas` legt die Namen im Container fest, `grid-area` ordnet ein Item einem Grid Area zu. Das Seitenlayout der Toolbox hat die Grid Areas `header`, `main` und `footer`.

Siehe auch: [Grid](#), [Landmark](#)

Header-Zelle

Definition: Zelle einer Tabelle, die eine Spalte oder eine Zeile benennt: das Element `<th>`. Das Attribut `scope` nennt die Richtung (`col`, `row`). Im Accessibility Tree hat sie die Rolle `columnheader` oder `rowheader`; eine Datenzelle (`<td>`) hat die Rolle `cell`.

Siehe auch: [Caption](#), [Accessibility Tree](#)

Kaskade

Definition: Verfahren, mit dem der Browser aus mehreren Deklarationen derselben Eigenschaft für ein Element eine auswählt. In den Materialien in drei Schritten: Ebene, Spezifität, Reihenfolge im Quelltext.

Siehe auch: [Cascade Layers](#), [Spezifität](#), [MDN Web Docs](#)

Klasse

Definition: Selbst gewählter Name im Attribut `class` eines Elements (`class="code"`). Der Selektor `.code` wählt jedes Element mit dieser Klasse aus. Der Name nennt die Aufgabe, nicht das Aussehen.

Siehe auch: [Selektor](#), [Komponente](#), [Spezifität](#), [MDN Web Docs](#)

Komponente

Definition: Wiederverwendbarer Baustein einer Seite mit eigenem HTML-Aufbau und eigenen Regeln im Stylesheet, ausgewählt über eine Klasse (`.code`). Der Name der Klasse nennt die Aufgabe, nicht das Aussehen.

Siehe auch: [Pattern](#), [Klasse](#)

Kontrast

Definition: Verhältnis der Helligkeit zweier Farben zwischen 1:1 und 21:1, berechnet nach WCAG. Text braucht zum Hintergrund mindestens 4,5:1, großer Text (ab 24 px oder ab 18,66 px fett) 3:1. Die DevTools nennen den Wert im Tooltip des **Inspect mode**.

Siehe auch: [oklch\(\)](#), [W3C Web Accessibility Initiative](#)

Label

Definition: Das Element `<label>`: der sichtbare Text zu einem Feld eines Formulars. Über `for` und `id` ist er der Name des Felds im Accessibility Tree; ein Klick auf das Label setzt den Fokus in das Feld. Das Attribut `name` ist dagegen der Schlüssel des Werts in den gesendeten Daten.

Siehe auch: [Placeholder](#), [Accessibility Tree](#)

Landmark

Definition: Seitenbereich mit fester Rolle im Accessibility Tree, zu dem Screenreader direkt springen können. HTML-Elemente → Rollen: `<header>` → `banner`, `<nav>` → `navigation`, `<main>` → `main`, `<footer>` → `contentinfo` (header/footer nur als direkte Kinder von `<body>`).

Siehe auch: [Accessibility Tree](#)

Layout Shift

Definition: Verschiebung von sichtbarem Inhalt, nachdem die Seite dargestellt ist, zum Beispiel weil ein Bild ohne reservierte Fläche lädt. Die Attribute `width` und `height` am `<img>` reservieren die Fläche vor dem Laden.

Siehe auch: [Lazy Loading](#), [MDN Web Docs](#)

Lazy Loading

Definition: Aufgeschobenes Laden: Mit `loading="lazy"` fordert der Browser eine Bilddatei erst an, wenn das Bild in die Nähe des Viewports kommt. Wirkt nur auf Seiten, die mehrere Viewports lang sind.

Siehe auch: [Layout Shift](#), [Viewport](#), [web.dev](#)

light-dark()

Definition: CSS-Funktion mit zwei Farben: Sie liefert im Light Mode die erste und im Dark Mode die zweite. Sie wirkt nur mit `color-scheme` und wird an dem Element ausgewertet, dessen Deklaration die Farbe verwendet.

Siehe auch: [Color Scheme](#), [Token](#), [MDN Web Docs](#)

Linktext

Definition: Der Text zwischen `<a>` und `</a>`. Er ist der Name des Links im Accessibility Tree und nennt das Ziel so, dass er ohne den umgebenden Satz verständlich ist.

Siehe auch: [Accessibility Tree](#)

Main Navigation

Definition: Die Navigation im Header jeder Seite mit den Zielen der Website. Enthält eine Seite mehrere Elemente `<nav>`, erhält jedes über `aria-label` einen Namen; in der Toolbox der Studierenden trägt sie den Namen „Main“.

Siehe auch: [Landmark](#), [Breadcrumbs](#)

Margin Collapsing

Definition: Zusammenfallen der Margins zweier Block-Boxen untereinander: Der Abstand ist der größere der beiden Werte, nicht ihre Summe. Padding und Rahmen fallen nie zusammen.

Siehe auch: [Box Model](#), [MDN Web Docs](#)

Markdown

Definition: Textformat, das die Struktur eines Textes (Überschriften, Listen, Links, Code, Tabellen) mit Zeichen im Text kennzeichnet. Das Skript folgt CommonMark, Tabellen stammen aus GitHub Flavored Markdown (GFM).

Siehe auch: [README](#), [Code-Block](#), [CommonMark Spec 0.31.2](#)

Media Query

Definition: Bedingung im Stylesheet (@media), unter der Regeln gelten, zum Beispiel ab einer Breite des Viewports. Ein Grid mit `repeat(auto-fill, minmax())` passt die Spaltenzahl ohne Media Query an den Platz an.

Siehe auch: [Breakpoint](#), [Mobile-first](#), [Container Query](#), [Grid](#), [Viewport](#)

Merge

Definition: Zusammenführen der Commits eines Branch in einen anderen Branch. Ein Merge-Commit hat zwei Vorgänger: den letzten Commit beider Branches.

Siehe auch: [Branch](#), [Merge-Konflikt](#), [Pull Request](#)

Merge-Konflikt

Definition: Zustand, in dem Git zwei Änderungen an derselben Stelle einer Datei nicht automatisch zusammenführen kann. Git schreibt beide Fassungen mit Konfliktmarkern in die Datei; eine Person entscheidet über den Inhalt.

Siehe auch: [Merge](#)

Mobile-first

Definition: Reihenfolge im Stylesheet: Die Regeln ohne Bedingung beschreiben die schmale Form der Seite. Jede Media Query ergänzt Regeln ab einem Breakpoint und steht nach ihnen.

Siehe auch: [Media Query](#), [Breakpoint](#)

oklch()

Definition: CSS-Funktion für eine Farbe aus Helligkeit (0 % bis 100 %), Chroma (0 für Grau bis etwa 0,4) und Farbton (Winkel von 0 bis 360). Die Helligkeit folgt der Wahrnehmung: Farben mit gleicher Helligkeit haben zu einem Hintergrund einen ähnlichen Kontrast.

Siehe auch: [Kontrast](#), [MDN Web Docs](#)

Override-Datei (overrides.css)

Definition: Datei `assets/css/overrides.css` der Toolbox für eigene Gestaltung. Sie gehört zur Ebene `overrides`, die in `main.css` zuletzt steht; eine Regel darin gewinnt gegen die Regeln aller anderen Dateien. Eigene Gestaltung ist freiwillig. Die Kapitel ändern diese Datei nicht.

Siehe auch: [Cascade Layers](#), [Custom Property](#)

Pattern

Definition: Bewährte, wiederverwendbare Lösung für ein wiederkehrendes Problem – hier: ein Baustein einer Webseite (z. B. Breadcrumbs, Landmarks) mit festem HTML-Aufbau und bekannten Regeln zur Barrierefreiheit.

Siehe auch: [Toolbox](#)

Pfad

Definition: Weg zu einer Datei. **Relativ** (`../assets/img/logo.svg`) von der Datei aus, in der er steht – funktioniert, wo auch immer das Projekt liegt. **Absolut** (`C:\Users\...\Users/...`) vom obersten Ordner aus (Windows: Laufwerk, macOS: /) – funktioniert nur auf einem Rechner. In HTML und CSS für Dateien des eigenen Projekts immer relativ und immer mit /. Ein Link auf eine andere Website enthält eine absolute URL mit Schema (`https://git-scm.com/`); sie ist kein Pfad auf dem eigenen Rechner.

Pflichtfeld

Definition: Feld eines Formulars mit dem Attribut `required`. Der Browser sendet das Formular nur ab, wenn das Feld einen Wert enthält, und zeigt sonst eine Fehlermeldung.

Siehe auch: [Validierung](#)

Placeholder

Definition: Das Attribut `placeholder`: ein Beispieltext in einem leeren Feld. Er verschwindet mit der ersten Eingabe und ersetzt kein Label.

Siehe auch: [Label](#)

Popover

Definition: Element mit dem Attribut `popover`. Der Browser stellt es nur dar, solange es offen ist; offen liegt es über allen anderen Elementen der Seite. Ein Button mit `popover-target` öffnet und schließt es, Escape und ein Klick daneben schließen es. Die Pseudoklasse `:popover-open` trifft ein offenes Popover.

Siehe auch: [:has\(\)](#), [@starting-style](#), [MDN Web Docs](#)

Progressive Enhancement

Definition: Vorgehen, bei dem eine Seite mit den Features funktioniert, die jeder Browser unterstützt, und neuere Features sie in Browsern mit Unterstützung verbessern. Ein Feature ohne Unterstützung fällt ohne Schaden weg oder erhält einen Ersatz mit `@supports`. Der englische Begriff wird nicht übersetzt.

Siehe auch: [Baseline](#), [@supports](#), [MDN Web Docs](#)

Pull

Definition: Abruf neuer Commits von einem Remote und Übernahme in den aktuellen Branch mit `git pull`, in VS Code **Git: Pull**. Pull besteht aus Fetch und Merge.

Siehe auch: [Fetch](#), [Push](#)

Pull Request

Definition: Anfrage auf einer Plattform wie GitHub, die Commits eines Branch in einen anderen Branch zu übernehmen, meist nach `main`. Zeigt alle Änderungen und sammelt Kommentare, Reviews und Freigaben.

Siehe auch: [Review](#), [Merge](#), [Branch](#)

Push

Definition: Übertragen der lokalen Commits zu einem Remote mit `git push`, in VS Code **Sync Changes**. Danach zeigt etwa GitHub denselben Stand.

Siehe auch: [Remote](#), [Commit](#)

Query Container

Definition: Element mit der Deklaration `container-type`, dessen Breite eine Container Query abfragt. Der Browser berechnet seine Breite ohne den Inhalt. Der Begriff unterscheidet das Element vom Flex-Container und vom Grid-Container.

Siehe auch: [Container Query](#), [Container](#) und [Item](#)

README

Definition: Datei `README.md` im obersten Ordner eines Projekts. Sie beschreibt Zweck, Aufbau und Start des Projekts für Personen, die es nicht angelegt haben; GitHub zeigt sie auf der Startseite des Repositorys.

Siehe auch: [Markdown](#), [Repository](#)

Regel

Definition: Baustein eines Stylesheets aus einem Selektor und einem Block von Deklarationen in geschweiften Klammern: `figcaption { color: var(--color-muted); }`.

Siehe auch: [Selektor](#), [Deklaration](#), [CSS](#)

Remote

Definition: Verbundene Kopie eines Repositorys auf einem anderen Rechner, meist einer Plattform wie GitHub. Standardname `origin`; `git push` überträgt Commits dorthin.

Siehe auch: [Repository](#)

Repository

Definition: Ein von Git verwaltetes Projekt: Arbeitsordner plus die gesamte Versionsgeschichte im Ordner `.git`. In das Repository gelangt, was ein Commit enthält; die Commits liegen im Ordner `.git`.

Siehe auch: [Commit](#), [Git](#)

Responsive Images

Definition: Mehrere Dateien für ein Bild, aus denen der Browser eine wählt. In der Toolbox: `<picture>` mit `<source media>` liefert je nach Breite des Viewports eine andere Datei; dargestellt wird immer das `<img>`.

Siehe auch: [Viewport](#), [Alt-Text](#), [MDN Web Docs](#)

Review

Definition: Prüfung eines Pull Request durch eine andere Person vor dem Merge. Ergebnis ist ein Kommentar, eine Freigabe (**Approve**) oder die Bitte um Änderungen (**Request changes**).

Siehe auch: [Pull Request](#)

Selektor

Definition: Teil einer Regel, der die Elemente auswählt, für die die Regel gilt: ein Element (a), eine Klasse (.code), ein Attribut ([aria-current="page"]), eine Pseudoklasse (:focus-visible) oder eine Kombination (nav a:jedes <a> in <nav>; main > p:jedes <p> direkt in <main>; * + *:jedes Element direkt nach einem anderen). Eine Liste mit Komma (main > * + *, article > * + *) gilt für jeden ihrer Selektoren.

Siehe auch: [Spezifität](#), [Deklaration](#)

Sidebar

Definition: Spalte neben dem Hauptinhalt. In der Toolbox enthält sie die Navigation mit allen Patterns (<nav aria-label="Patterns">). Ab 64rem steht sie als Spalte neben dem Hauptinhalt; darunter öffnet der Button „Patterns“ sie als [Popover](#).

Siehe auch: [Main Navigation](#), [Breakpoint](#)

Skip-Link

Definition: Link am Anfang jeder Seite, der zum Hauptinhalt führt (href="#inhalt"). Er ist der erste [Tab-Stopp](#) und erscheint erst mit dem Fokus.

Siehe auch: [Landmark](#), [W3C Web Accessibility Initiative](#)

Spezifität

Definition: Gewicht eines Selektors aus drei Zahlen: IDs, dann Klassen, Attribute und Pseudoklassen, dann Elemente und Pseudoelemente (nav ol a = 0,0,3; .code = 0,1,0). Sie entscheidet zwischen zwei Regeln derselben Ebene.

Siehe auch: [Kaskade](#), [Selektor](#), [MDN Web Docs](#)

Staging Area

Definition: Zwischenbereich in Git für Änderungen, die in den nächsten Commit sollen (git add). Erlaubt, zusammengehörige Änderungen gezielt zu einem Commit zu bündeln.

Siehe auch: [Commit](#)

Stylesheet

Definition: Datei mit CSS-Regeln, meist mit der Endung .css. Eine HTML-Seite lädt sie mit <link rel="stylesheet" href="...">; findet der Browser die Datei nicht, erscheint die Seite ohne Gestaltung. Regeln für nur eine Seite stehen im Element <style>, Deklarationen für nur ein Element im Attribut style.

Siehe auch: [Pfad](#), [Selektor](#), [CSS](#)

Tab-Stopp

Definition: Element, das der Fokus mit Tab erreicht: Links, Buttons, Felder und Elemente mit `tabindex="0"`. Eine Gruppe von Radio Buttons ist ein Tab-Stopp; die Pfeiltasten wählen darin.

Siehe auch: [Skip-Link](#), [W3C Web Accessibility Initiative](#)

Tag

Definition: Auszeichnung in spitzen Klammern, die ein Element beginnt (`<title>`) oder beendet (`</title>`). Element und Tag sind nicht dasselbe: Das Element umfasst beide Tags und den Inhalt dazwischen.

Siehe auch: [Element](#)

Token

Definition: Custom Property für einen Wert der Gestaltung: Farbe, Schrift, Schriftgröße, Zeilenhöhe, Abstand oder Maß. Der Name nennt die Aufgabe, nicht das Aussehen (`--color-paper`, nicht `--color-white`). Die Toolbox definiert alle Tokens in `tokens.css`; das Pattern heißt „Design-Tokens“.

Siehe auch: [Custom Property](#), [Design Tokens Community Group](#)

Transition

Definition: Übergang einer Eigenschaft von einem Wert zu einem anderen über eine Dauer, festgelegt mit `transition`. Eigenschaften ohne Zwischenwerte wie `display` wechseln nur mit `allow-discrete` erst am Ende der Dauer. Personen, die im System Bewegung reduziert haben, erhalten in der Toolbox keine Transition (`prefers-reduced-motion`).

Siehe auch: [@starting-style](#), [MDN Web Docs](#)

Überschriften-Hierarchie

Definition: Gliederung einer Seite durch `<h1>` bis `<h6>`: genau ein `<h1>`, keine übersprungene Ebene, Ebene nach Struktur statt nach Schriftgröße.

Siehe auch: [Accessibility Tree](#)

Validierung

Definition: Prüfung der Eingaben eines Formulars durch den Browser vor dem Absenden, gesteuert über Attribute wie `required`, `type` und `pattern`. Der Selektor `:invalid` trifft auf ein Feld mit ungültigem Wert zu, `:user-invalid` erst nach einer Eingabe der Person oder nach dem Absenden.

Siehe auch: [Pflichtfeld](#)

Vererbung

Definition: Ein Element übernimmt den Wert einer Eigenschaft vom umgebenden Element, solange keine Regel die Eigenschaft für das Element selbst festlegt. Custom Properties, Textfarbe und Schrift werden vererbt, Abstände und Hintergrund nicht. Eine Custom Property wird mit ihrem berechneten Wert vererbt: `var()` ist darin eingesetzt, eine Funktion wie `light-dark()`

bleibt stehen.

Siehe auch: [Custom Property](#), [Kaskade](#), [MDN Web Docs](#)

Viewport

Definition: Der sichtbare Bereich der Seite im Browserfenster. Ohne `<meta name="viewport">` rechnen Mobilbrowser mit einer virtuellen Breite von ca. 980 px und verkleinern die Seite.

WCAG

Definition: Web Content Accessibility Guidelines des W3C, aktuell in der Fassung 2.2: vier Prinzipien, 13 Richtlinien und Erfolgskriterien mit den Stufen A, AA und AAA. BFG und BITV 2.0 verlangen über EN 301 549 die Stufe AA.

Siehe auch: [Erfolgskriterium](#), [Kontrast](#), [W3C Web Accessibility Initiative](#)

Wordmark

Definition: Name einer Website als gestalteter Schriftzug, meist oben links im Header und als Link auf die Startseite. In der Toolbox ist es der Link „Toolbox“ mit dem farbigen Quadrat.

Siehe auch: [Landmark](#)

Workspace

Definition: Ein in VS Code geöffneter Ordner. Explorer, Pfad-Vervollständigung und Terminal beziehen sich auf diesen Ordner.

Zeichenkodierung

Definition: Regel, nach der ein Programm die gespeicherten Bytes einer Datei in Zeichen übersetzt. UTF-8 enthält alle Zeichen, auch Umlaute wie „ä“ und „ß“. Eine HTML-Seite nennt sie mit `<meta charset="utf-8">`.

Siehe auch: [MDN Web Docs](#)